队伍编号	MCB2602551
赛道	B

基于多源数据融合与多尺度建模的泥石流风险预测及动力学模拟

摘　要

本文以 2026 年 8 月 26 日西藏吉隆泥石流灾害为研究背景，综合利用气象、辐射、土壤、湍流通量、冰川遥感及数字高程模型等多源数据，按照“灾前因素识别—触发风险评价—时空动态预警—灾害运动模拟”的研究思路，构建多尺度泥石流灾害分析与预测模型，实现从环境异常识别到灾害运动过程模拟的系统研究。

针对问题一，建立基于**多时间窗口异常检测**与**多指标综合评价**的关键因素识别模型。首先对 MET、RADM、SOIL、FLUX 四类逐小时数据进行质量控制与时间匹配，构建灾前 6、24、48、72 小时特征窗口；随后结合历史同期对照、暖季经验百分位、稳健异常评分及变量相关性分析，从气象和地表能量交换指标中筛选主要关联因素。初步分析表明，**正积温**、**水汽压亏缺**（VPD）、**净辐射**、**感热通量**和**土壤热通量**是与本次灾害前环境变化关联较强的五类因素，反映了水热背景与地气能量交换的综合作用。

针对问题二，建立累计降雨量与正积温耦合的**逻辑回归风险评价模型**，并利用冰川厚度、冰流速度及坡度进行空间辅助筛查。以 72 小时累计降雨量和 72 小时正积温为核心特征，采用时间前向划分、类别平衡及训练期阈值选择，构建泥石流记录事件日的经验判别模型。结果表明，P72+D72 组合在测试集上的 **AUC 为 0.7132**，事件日召**回率为 66.67%**。在此基础上推导降雨—正积温联合判别边界，为泥石流风险初步筛查提供定量依据。

针对问题三，构建融合 48 小时风险情景、条件触发时间区间和地形汇流分析的时空预警模型。利用第二问的逻辑回归评分，通过危险率近似构建未来 48 小时风险指标，并采用条件时间分布生成名义 95%触发时间预测区间；针对历史灾害仅记录发生日期的问题，引入区间删失评价方法。测试期共识别 3 个记录事件日中的 2 个，**事件日检出率为 66.67%**，背景窗口报警比例为 32.50%。进一步结合 DEM 与 **D8 汇流算法**开展空间路径分析，获得两条候选汇流路径，长度分别约为 **26.24km** 和 **28.00km**。

针对问题四，建立基于 **Voellmy 摩擦阻力**与**侵蚀夹带耦合**的二维深度平均泥石流动力学模型。利用真实 DEM 构建地形计算网格，以质量守恒和动量守恒方程描述泥石流运动，并引入分阶段摩擦阻力、速度阈值侵蚀和低速沉积机制，采用**有限体积法**及 **Rusanov 数值**通量进行动态求解。通过文献运动特征约束和参数组合试算，完成 600s 泥石流运动模拟。结果显示，模拟约在 440s 进入吉隆口岸 300m 邻域，全域瞬时最大格网流速为 **86.99m/s**，累计影响面积约 **31.56km²**，侵蚀体积约 **5852 万 m³**。结合三维地形可视化，展示了泥石流沿复杂沟谷传播及侵蚀沉积的动态演化过程。

关键词：泥石流灾害；多源数据融合；逻辑回归；时空预警；Voellmy 动力学模型；

目　录

一、问题重述 1
1.1 问题背景 1
1.2 目标任务 1
二、模型假设与符号说明 2
2.1 模型假设 2
2.2 符号说明 2
三、问题分析 3
3.1 问题一分析 3
3.2 问题二分析 3
3.3 问题三分析 4
3.4 问题四分析 4
四、问题一的建模与求解 4
4.1 数据预处理与质量控制 4
4.1.1 数据范围与事件窗口 4
4.1.2 数据质量控制与时间序列融合 5
4.2 特征工程与关联性分析方法 6
4.2.1 多时间窗口统计 6
4.2.2 历史暖季背景构建与经验百分位计算 6
4.2.3 综合评分模型 7
4.3.1 灾前关键气象异常 8
五、问题二的建模与求解 11
5.1 数据来源与补充数据处理 11
5.1.1 历史泥石流空间数据 11
5.1.2 补充历史灾害日期数据 12
5.1.3 气象观测数据 12
5.1.4 冰川数据 13
5.2 动态气象触发模型建立 13
5.2.1 累计降雨指标 13
5.2.2 正积温指标 13
5.2.3 特征标准化 14
5.2.4 逻辑回归评分模型 14
5.2.5 报警评分分界 14
5.3 模型求解与实验设计 15
5.3.1 时间划分 15

5.3.2 候选模型设置 16
5.3.3 模型评价指标 16
5.4 模型求解结果 17
5.4.1 不同降雨窗口模型比较 17
5.4.2 P_{72}+D_{72}模型参数 17
5.4.3 经验联合临界条件 18
5.4.4 测试集判别结果 19
5.5 冰川静态条件分析 19
5.5.1 冰川静态变量统计 19
六、问题三的建模与求解 20
6.1 48 小时气象风险预警模型 20
6.1.1 气象风险因子 20
6.1.2 冻结的第二问逻辑回归模型 20
6.1.3 未来 48 小时事件风险情景估计 21
6.1.4 预警触发规则 21
6.2 名义 95%触发时间预测区间 21
6.2.1 条件触发时间分布 21
6.2.2 2026 年典型事件的预测结果 22
6.3 基于区间删失的时间区间评价 22
6.3.1 严格覆盖率上下界 22
6.3.2 模糊加权覆盖率 23
6.3.3 模糊覆盖率计算结果 23
6.4 预警时间与提前量计算 23
6.4.1 预警决策模型 23
6.4.2 2021 年独立测试事件的预警结果 24
6.5 泥石流空间传播路径分析 24
6.6 模型对比与最终选择 24
七、第四问模型建立与求解 25
7.1 研究区地形与初始条件构建 25
7.1.1 地形数据及坐标统一 25
7.1.2 初始释放区域和物源厚度 25
7.2 二维深度平均动力学模型 26
7.2.1 基本假设与守恒变量 26
7.2.2 水平动量守恒 27
7.2.3 Voellmy 摩擦阻力 27
7.3 侵蚀、沉积与床面变化耦合 27

7.3.1 速度阈值侵蚀模型.....27
7.4 参数确定及数值计算方法.....28
7.4.1 文献约束与参数筛选.....28
7.4.2 有限体积数值离散.....29
7.5 泥石流运动过程与三维展示.....30
7.5.1 二维时空演化.....30
7.5.2 三维地形动态展示.....30
7.6 运动强度及侵蚀沉积结果.....31
7.6.1 峰值流深与流速.....31
7.6.2 侵蚀沉积空间分布.....31
八、模型评价与讨论.....32
九、参考文献.....33
十、 附录.....34
1.. 软件说明.....34
2. 命令内容.....34
2.1 使用记录 A01：赛题理解辅助.....34
2.2 使用记录 A02：数值求解辅助.....35
2.3 使用记录 A03：代码调试辅助.....35
2.4 使用记录 A04：图表与流程图生成.....35
2.5 使用记录 A05：语言润色辅助.....36

一、问题重述

1.1 问题背景

泥石流是高山峡谷地区典型的突发性地质灾害，其形成与演化受到降水、冰雪融水、松散物源及地形地貌等多种因素的共同影响。在重力作用下，水体与松散固体物质形成高流动性混合体，沿沟谷快速运移，具有突发性强、演化过程复杂、传播速度快及破坏范围广等特点。尤其在高海拔冰川发育地区，冰岩崩塌、冰雪融水和沟道侵蚀等过程可能相互耦合，形成由初始失稳向下游泥石流、山洪逐步演化的链式灾害[12]，进一步增加了灾害风险识别与动态预测的难度。

西藏日喀则市吉隆县位于喜马拉雅山脉中段，区域内地形起伏剧烈、沟谷深切，冰川及松散堆积物广泛分布，具有复杂的地质环境与水热条件。2026 年 8 月 26 日，吉隆地区发生重大冰岩崩—泥石流灾害，灾害物质由高海拔源区快速下泄，在沿程侵蚀、物质裹挟及水体掺混等作用下，逐步演化为具有较强破坏性的泥石流，并对吉隆口岸及下游区域造成严重影响[1]。此次事件具有明显的多因素耦合和灾害链演化特征，为研究高山峡谷地区泥石流的形成条件、触发机制及动力传播过程提供了典型案例。

针对该类灾害，单一降雨指标难以全面反映复杂水热条件与冰川环境的共同作用，而不同来源的气象观测、遥感影像及地形数据又存在时间分辨率、空间尺度和数据特征不一致等问题。因此，如何有效融合多源时空数据，识别灾害发生前的关键环境变化，建立合理的触发风险评价与动态预警模型，并结合真实地形模拟泥石流的运动路径、速度变化及影响范围，成为高山峡谷地区泥石流灾害风险研究的重要问题。基于此，本文以吉隆“8·26”泥石流灾害为研究对象，开展灾前影响因素分析、触发风险评价、时空预警及泥石流动力学模拟研究，为区域灾害风险识别与防灾减灾提供定量参考。

1.2 目标任务

围绕 2026 年 8 月 26 日吉隆沟泥石流事件，结合多源气象、遥感及地形数据，建立泥石流风险评估与预测模型，完成以下四项任务：

问题一：分析灾害发生前的气象异常、大气边界层结构及极端天气过程，筛选并排序关联性最高的五个关键因素。

问题二：识别泥石流发生的关键触发条件及临界阈值，建立风险评价模型，并分析模型的时间与空间可靠性。

问题三：建立动态预警模型，预测泥石流可能发生的时间及其 95%置信区间，评估预警提前量、区间覆盖率和空间预警范围。

问题四：结合 DEM 及相关地形参数，建立泥石流运动模拟模型，分析其运动路径、速度变化和可能影响范围。

二、模型假设与符号说明

2.1 模型假设

为统一全文口径，模型假设和符号集中列示如下；其余结论均按这些定义解释。假设仅限定模型解释范围，不视为经数据证实的物理定律。

假设 1：同一日期的多条灾害记录在时间分析中合并为一个记录事件日；日期级记录只表示事件发生在该日，不据此指定具体小时，也不将无记录日认定为无灾日。

假设 2：累计降雨量和正积温作为气象触发条件的经验代理变量；二者与灾害记录的统计关联不直接等同于物理因果关系，正积温也不等于实际融水量。

假设 3：预警模型在发布时刻只使用当时可获得的信息；没有独立未来气象预报时，未来 48 小时风险仅作为条件情景，不视为真实天气过程。

假设 4：逻辑回归系数及报警阈值仅由训练期确定，测试期不参与拟合、阈值选择或调参。

假设 5：D8 路径仅表示基于地形的候选汇流通道，不等同于经现场验证的泥石流运动轨迹。

假设 6：冰川厚度、坡度、速度等静态或多年气候态指标仅用于空间辅助筛查，不解释为事件发生前的实时形变或冰崩概率。

2.2 符号说明

表 1 符号及变量说明

符号	含义	单位
t	每日零时的评估时刻	h
P_h	第 h 个小时的降雨量	mm
T_h	第 h 个小时的气温	℃
$P_{24}(t)$	评估时刻前 24 小时累计降雨量	mm
$P_{72}(t)$	评估时刻前 72 小时累计降雨量	mm
$P_{360}(t)$	评估时刻前 360 小时累计降雨量	mm
$D_{72}(t)$	评估时刻前 72 小时累计正积温	℃·日
y_t	日期标签，记录事件日为 1，背景日为 0	无量纲
X_{tj}	日期 t 的第 j 个模型特征	依特征而定
μ_j	训练集第 j 个特征的均值	与特征相同
σ_j	训练集第 j 个特征的标准差	与特征相同
Z_{tj}	标准化后的特征	无量纲
β_0	逻辑回归截距	无量纲

符号	含义	单位
β_j	标准化特征的回归系数	无量纲
η^t	逻辑回归线性评分	无量纲
st	Sigmoid 变换后的模型评分	0—1
s_c	训练期确定的报警评分分界	0—1
C	逻辑回归正则化参数	无量纲
H	冰川厚度	m
V	冰流速度	m/年
θ	冰川坡度	°

三、问题分析

3.1 问题一分析

问题一要求基于 MET、RADM、SOIL、FLUX 等观测数据，分析吉隆县及周边高寒山区的大气边界层结构、天气过程和极端气象事件与 2026 年 8 月 26 日泥石流事件的关联性，并筛选五个特征因素[14]。本文将“关联”限定为灾前观测特征相对于历史背景的偏离及其时间一致性，不将单次事件中的同步变化解释为因果效应。

本问按“数据质量控制—多时间窗口比较—严格同期对照—暖季背景对照—变量相关性检查—方向修正评分”展开。首先统一 MET、RADM、SOIL、FLUX 的时间索引与变量单位，并核查传感器 QC 缺失和异常值；其次构造灾前 6、24、48、72h 窗口统计量，分别与 2021—2025 年相近日期的同长度窗口及同年暖季滚动窗口比较，以区分短时偏离和季节背景位置；随后检查候选变量间相关性，降低同类信息重复计权；最后按预先说明的方向与机制评分排序。结果按天气过程、边界层结构和极端能量条件三类归纳。由于只有单次目标事件，所得是候选关联因素而非因果贡献排序；单站观测也不能独立确认冰川失稳、水源或沟道过程。

3.2 问题二分析

泥石流是降雨、地形、松散物源及水热条件等多种因素共同作用的结果。在吉隆县高山峡谷及冰川发育地区，不同泥石流灾害的触发机制存在明显差异。部分泥石流主要受到短时或持续降雨影响，部分可能与冰雪融水及前期水分积累有关，而冰崩、冰湖溃决等过程还涉及冰体稳定性、湖泊蓄水条件和物质运动过程[2]。

因此，若直接使用单一降雨阈值描述所有泥石流事件，可能忽略不同机制之间的差异；但若分别为各种机制建立独立的监督学习模型，又会受到历史事件日期和机制标签不足的限制。

针对上述问题，本文采用动态气象触发评估与静态冰川条件分析相结合的分层建模方法。首先，根据补充收集的历史灾害日期和逐小时气象观测，提取累计降雨量与正积温指标，建立逻辑回归模型，分析水分输入和热力条件对记录事件日的联合判别作用，并计算经验判别边界。其次，利用冰川厚度、冰流速度和坡度等静态数据，对潜在源区进行辅助筛查。最后，通过时间前向划分、不同降雨窗口的比较和空间覆盖率统计，说明模型的适用范围。动态模型输出为相对评分，不直接代表真实灾害概率；静态冰川指标也不能证明已经识别出冰崩触发阈值。

3.3 问题三分析

吉隆县泥石流风险受降水、气温、冰川状态及地形条件共同影响。第二问已基于 72 小时累计降水量和 72 小时正积温建立逻辑回归模型。第三问沿用该模型，进一步估计未来 48 小时风险情景，描述触发时间的不确定范围，并结合地形与冰川静态数据进行空间筛查。

本问分为三个相互衔接的环节：

1.根据预报发布时刻已知的气象信息，判断未来 48 小时是否需要发布泥石流风险预警。

2.构建泥石流触发时间的名义 95%条件预测区间，利用只有发生日期的历史事件进行区间删失评价。

3.结合 DEM、D8 汇流[10]方向及冰川地形数据，计算候选传播通道及其空间距离。

据此形成“气象风险判别—条件时间区间—区间删失检验—空间通道筛查”的分析流程。

3.4 问题四分析

问题四要求模拟吉隆沟泥石流的运动过程，并展示其路径、速度和影响范围。建模需要将 DEM 地形、泥石流初始条件与动力学过程结合起来[15]：首先构建地形计算网格并设置初始释放条件；随后采用深度平均模型描述流体运动，以 Voellmy 阻力刻画底床摩擦，并通过侵蚀夹带项描述流体与沟床物质的交换；最后提取不同时刻的流速、流深和影响范围并进行可视化。通过体积收支、数值稳定性和参数敏感性进行检查。

四、问题一的建模与求解

4.1 数据预处理与质量控制

4.1.1 数据范围与事件窗口

本文采用吉隆高寒山区 2021 年 1 月 1 日至 2026 年 8 月 27 日的逐小时地气相互作用综合观测资料，数据包括气象观测（MET）、辐射观测（RADM）、土壤环境观测（SOIL）和湍流通量观测（FLUX）四类，涵盖气温、降水、辐射收支、土壤水热状态及地气能量交

换等变量。通过统一时间索引与变量单位，对不同来源的观测数据进行整合，为灾前环境异常识别和关键影响因素筛选提供数据基础。

以 2026 年 8 月 26 日吉隆泥石流灾害为研究事件，按照赛题给定的 10:30 参考时刻，将灾前观测数据的截止时间设定为当日 10:00，保证所提取的灾前特征不包含分析截止时刻之后的信息。分别构建灾前 6h、24h、48h 和 72h 时间窗口，提取各变量的累计值、均值及变化特征，以分析不同时间尺度下的环境条件。

为识别灾前环境变化的异常程度，设置历史同期和暖季背景两类对照样本。其中，历史同期对照选取 2021—2025 年相同月、日、小时对应的等长时间窗口；暖季背景对照选取 2021—2025 年 6—9 月的等长时间窗口。通过比较事件窗口与两类历史背景的统计差异，综合分析灾前气象条件及地表能量交换过程的异常特征。

4.1.2 数据质量控制与时间序列融合

为保证多源观测数据的一致性与可靠性，首先将 MET、RADM、SOIL 和 FLUX 四类逐小时数据统一转换为 Datetime 时间索引，按照时间戳进行对齐与融合，并保留各变量的数据来源标识，构建统一的多变量时间序列。

在数据质量控制方面，将 9999.9、9999、−9999 及−9999.9 等异常占位值识别为缺失值。针对湍流通量数据，结合质量控制标识及观测条件进行筛选：当 QC_LE 或 QC_H 的质量标识值达到 8 及以上时[15]，将对应的潜热通量或感热通量观测值设为缺失；当摩擦风速 U*低于 0.1m/s 或 90%通量贡献区距离 FETCH90 小于 50m 时，对受影响的湍流通量观测进行质量筛查，以降低低湍流强度及观测代表性不足对分析结果的影响。

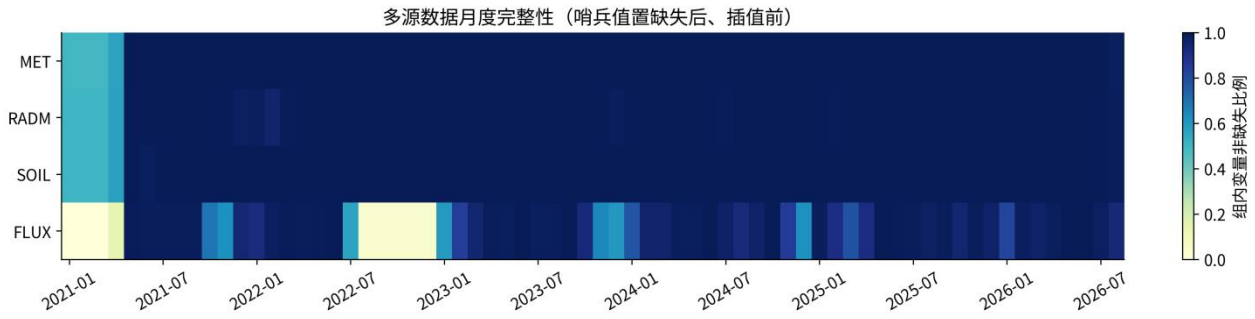


图 4-1 多源气象观测数据的月度完整性热力图（插值前）

对于连续缺失时间不超过 3h 的数据片段，采用线性插值进行补全；对于连续缺失时间超过 3h 的片段，保留缺失状态，避免长时间插值引入较大的估计偏差。同时，严格控制时间信息的使用范围，确保灾前特征提取仅依赖分析截止时刻及此前的有效观测数据。

经过上述处理，获得时间基准统一、变量来源清晰的多源观测数据，为后续多时间窗口统计、灾前异常检测及关键影响因素筛选提供数据基础。

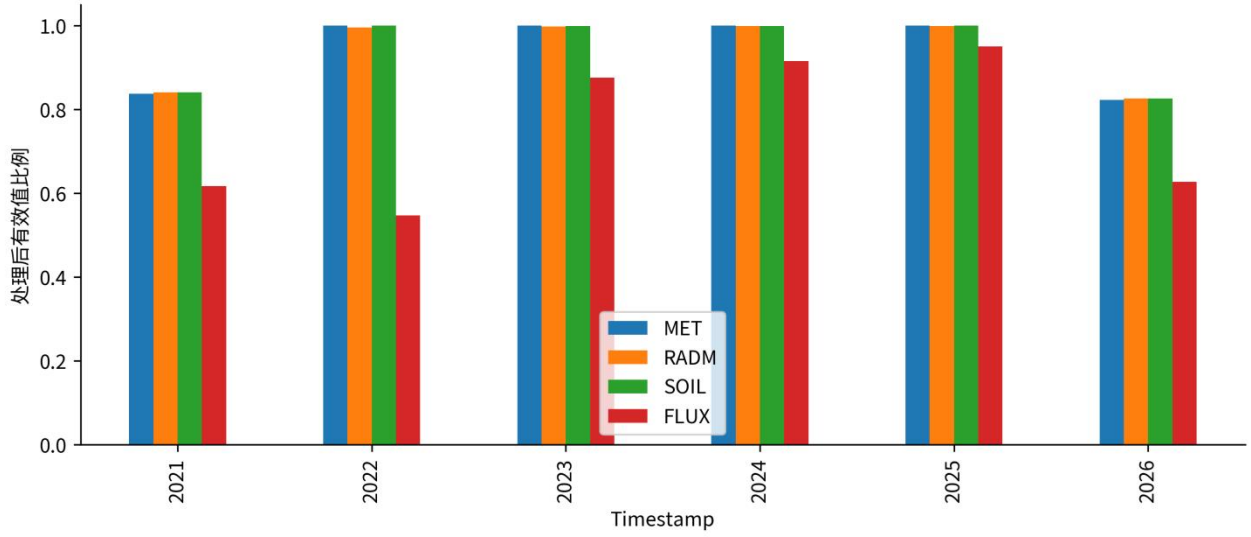


图 4-2 多源观测数据逐年有效保留率对比图

4.2 特征工程与关联性分析方法

4.2.1 多时间窗口统计

对变量v在窗口长度$h\epsilon\{6,24,48,72\}$下计算窗口均值：

$$\bar{x}_{v,h}(t) = \frac{1}{h}\sum_{k=0}^{h-1} x_v\,(t-k)$$

降水按窗口累计，其余变量按窗口均值统计。对比 6、24、48、72 小时结果，用于区分临近时段变化与较长时间的累积背景；具体窗口端点和单位沿用原始公式定义。

4.2.2 历史暖季背景构建与经验百分位计算

为衡量灾害发生前气象及地表能量交换特征的异常程度，选取 2021—2025 年每年 6—9 月的逐小时观测数据构建历史暖季背景样本。针对灾前 6h、24h、48h 和 72h 四种时间尺度，分别计算历史背景中的等长窗口统计量，使事件窗口与背景窗口具有一致的时间尺度和统计口径。

为提高统计结果的可靠性，设置窗口有效数据覆盖率阈值为 80%，仅保留有效观测比例不低于该阈值的背景窗口。历史背景样本独立于 2026 年目标事件数据构建，以避免目标事件信息影响参考分布。

严格同期对照以 2021—2025 年同月、同日、同小时结束的同长度窗口为参照，计算事件窗口相对于历史同期的稳健异常：

$$Z_{v,h}^{\mathrm{rob}} = \frac{\bar{x}_{v,h} - \mathrm{median}\left(\bar{x}_{v,h}^{\mathrm{hist}}\right)}{1.4826\,\mathrm{MAD}\left(\bar{x}_{v,h}^{\mathrm{hist}}\right)}$$

暖季背景对照以 2021—2025 年 6—9 月同长度窗口为参照，计算事件值在经验分布中的位置：

$$P_{v,h} = 100 \times \frac{1}{N} \sum_{i=1}^{N} I\left(\bar{x}_{v,h}^{\text{warm},i} < \bar{x}_{v,h}^{\text{event}}\right)$$

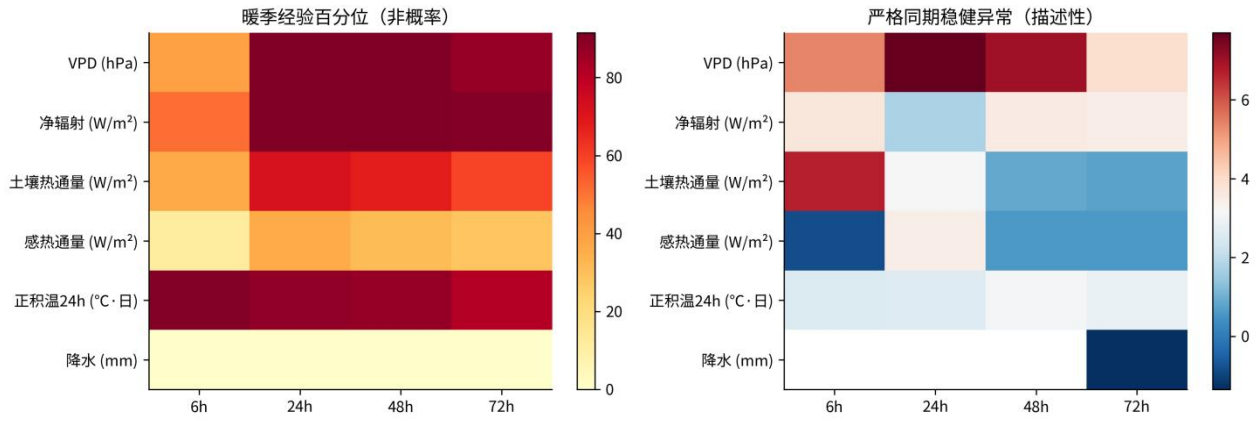


图 4-3 多时间窗口下的气象要素暖季百分位与稳健异常分析

严格同期对照回答事件窗口相对近年同一时段是否偏离；暖季背景对照回答其在季节内通常分布中的位置。由于同期年份有限，且暖季滚动窗口可能存在相关性，相关统计量仅作描述性比较。

4.2.3 综合评分模型

综合评分由参考文献[3]五个证据分量及权重，分别为数据质量、方向修正后的异常强度、多窗口一致性、变量独立性和物理机制解释；权重依次取 0.25、0.25、0.20、0.15、0.15。评分用于整理候选因素证据，不代表因果贡献率或灾害发生概率，且其敏感性尚需检验。

$$S_v = 0.25Q_v + 0.25P_v^* + 0.20C_v + 0.15I_v + 0.15M_v$$

其中，

$$P_v^* = P_v \delta_v$$

$$I_v = 1 - \max_{u \in \mathcal{R},\, u \neq v} |\rho_{vu}|$$

式中，Q_v为数据质量评分，P_v为暖季异常强度评分，δ_v为预先设定的方向一致系数，C_v为多窗口及不同背景基准下的一致性评分，I_v为相对于其他物理组代表变量的独立性评分，M_v为物理机制解释评分；$\mathcal{R}$表示参与独立性评价的物理组代表变量集合。各分量和综合得分按上述定义计算；提交前应以最终程序输出核对逐项分值及排序。

以正积温 24 小时为例：

$S_v = 0.25(1.0) + 0.25(0.492636) + 0.20(1.0) + 0.15(0.663317) + 0.15(0.9) = 0.807657$

按上述分项代入，正积温 24h 的综合评分为 0.807657。

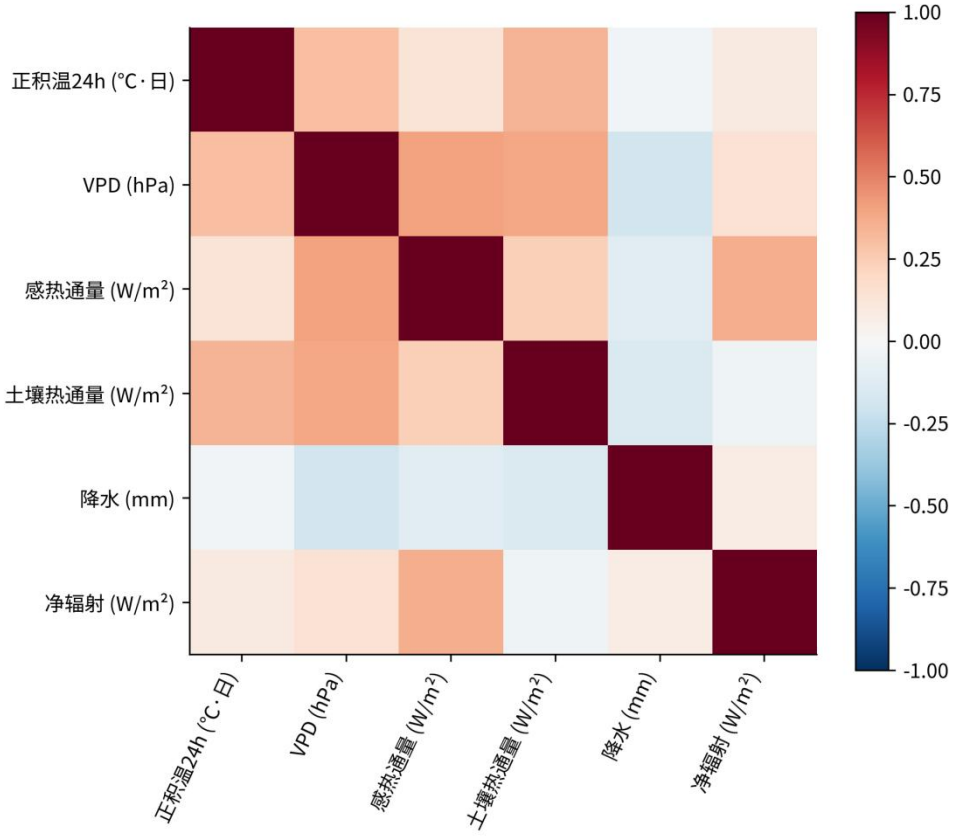


图 4-4 去日变化的 Spearman 相关系数热力图

方向一致系数δ_v按预先确定的规则取值：事件值方向与预期触发方向一致时取 1；方向相反但仍保留弱关联时取 0.2；事件窗口无该触发信号时取 0。该规则是评分设定，不是从本次事件数据估计出的物理常数。

变量按物理意义分为温度与融化背景、水汽条件、辐射与能量条件、边界层结构与湍流、降水过程、下垫面条件六组；组内优先按暖季异常强度评分选取代表变量。边界层组考虑到温度梯度暖季分布较窄，采用综合排序分值筛选代表变量：

$$S_v = 0.3P_v^* + 0.7M_v$$

代表变量集合 R 由正积温 24h、VPD、净辐射、感热通量、土壤热通量和降水构成；独立性评分 I_v在 R 内计算，以降低同组相关变量重复计权的影响。

4.3 模型求解与结果分析

4.3.1 灾前关键气象异常

为分析泥石流发生前研究区气象条件及地表能量交换过程的变化特征，选取正积温、水汽压亏缺（VPD）、净辐射、感热通量和土壤热通量五项指标，分别与历史严格同期背景及暖季背景进行对比。各指标的统计结果见表 2。

表 2 灾前关键环境因子与历史背景对比

环境指标	灾前统计值	同期中位数	暖季百分位	稳健异常分数
正积温 24h	313.22	281.81	81.1%	—

环境指标	灾前统计值	同期中位数	暖季百分位	稳健异常分数
水汽压亏缺 VPD	10.78hPa	8.03hPa	87.1%	1.42
净辐射	175.3W/m²	152.8W/m²	89.7%	1.31
感热通量	48.13W/m²	40.07W/m²	31.1%	—
土壤热通量	3.944W/m²	0.5664W/m²	59.2%	—

注：“—”表示现有统计材料未提供对应数值；正积温的统计时间窗口及单位需与最终计算定义统一。

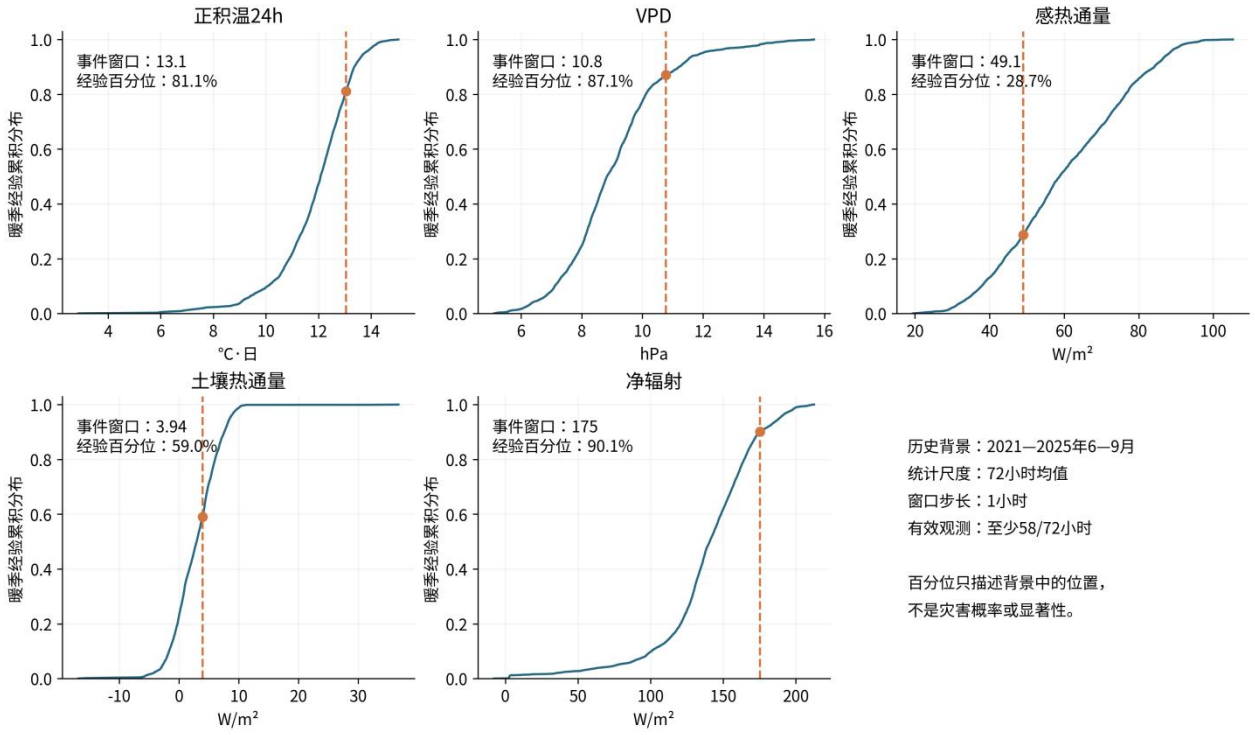


图 4-5 经验累积分布函数（ECDF）图

(1)温度与大气水汽条件分析

由表 2 可知，正积温指标为 313.22，高于历史严格同期中位数 281.81，暖季经验百分位达到 81.1%，反映出灾前研究区具有相对偏暖的环境特征。水汽压亏缺（VPD）为 10.78hPa，高于历史同期中位数 8.03hPa，暖季经验百分位为 87.1%，稳健异常分数为 1.42，表明灾前大气水汽亏缺程度处于历史暖季相对较高水平。

(2)辐射与地表能量交换分析

净辐射平均值为 175.3W/m²，高于历史严格同期中位数 152.8W/m²，暖季经验百分位为 89.7%，稳健异常分数为 1.31，表明灾前地表净能量输入相对较强，为研究区地表增温及潜在冰雪消融提供了有利的能量背景。

感热通量平均值为 48.13W/m²，高于历史严格同期中位数 40.07W/m²，但其暖季经验百分位仅为 31.1%。这表明该指标相对于相同日期的历史背景有所升高，但在整个暖季分布中

处于相对较低的位置。土壤热通量为 3.944W/m²，高于历史同期中位数 0.5664W/m²，暖季经验百分位为 59.2%，反映出地表与土壤之间的热量交换相对于严格同期背景有所增强。

(3) 灾前环境特征综合分析

综合上述结果，正积温、水汽压亏缺和净辐射在历史暖季背景下均表现出较高的经验百分位，反映出灾前研究区存在相对偏暖、水汽亏缺程度较高及地表净能量输入较强的环境特征。相比之下，感热通量和土壤热通量在历史同期与暖季背景下表现出不同程度的变化，可作为地气能量交换过程的辅助评价指标。

总体而言，灾前环境变化主要体现为温度、水汽及辐射条件的共同变化，为进一步开展关键影响因素综合评价与泥石流触发风险分析提供了统计依据。

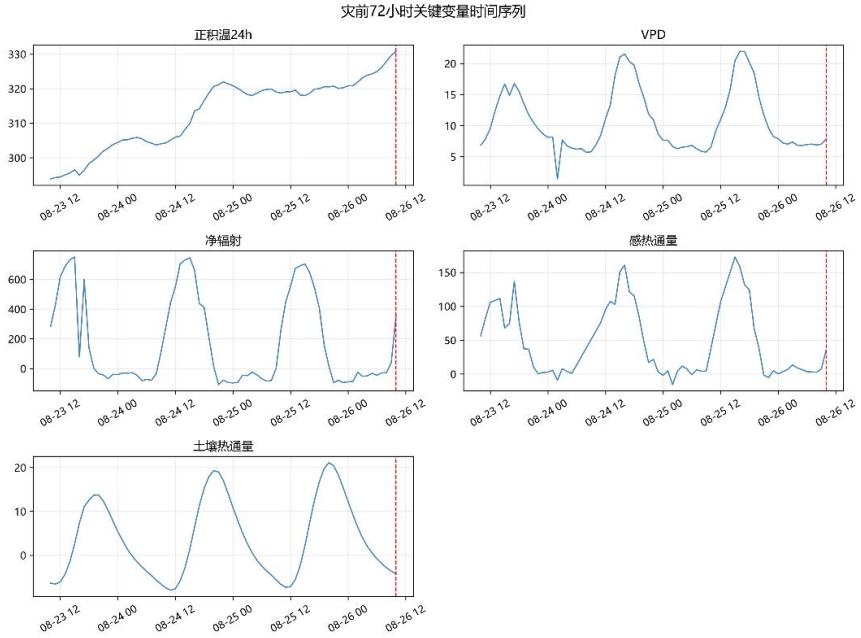


图 4-6 灾前 72 小时关键变量时间序列

4.3.2 前五因素识别结果及物理解释

综合异常程度、方向一致性及变量独立性分析，筛选出与灾前环境变化关联较强的五个关键因素：**正积温、水汽压亏缺（VPD）、净辐射、感热通量和土壤热通量**。其中，正积温反映灾前偏暖背景，VPD 表征大气水分状态，净辐射反映地表能量输入，感热通量和土壤热通量体现地气能量交换特征。上述因素从温度、水汽及能量收支三个方面揭示了灾前环境变化，为后续泥石流风险评价提供依据。

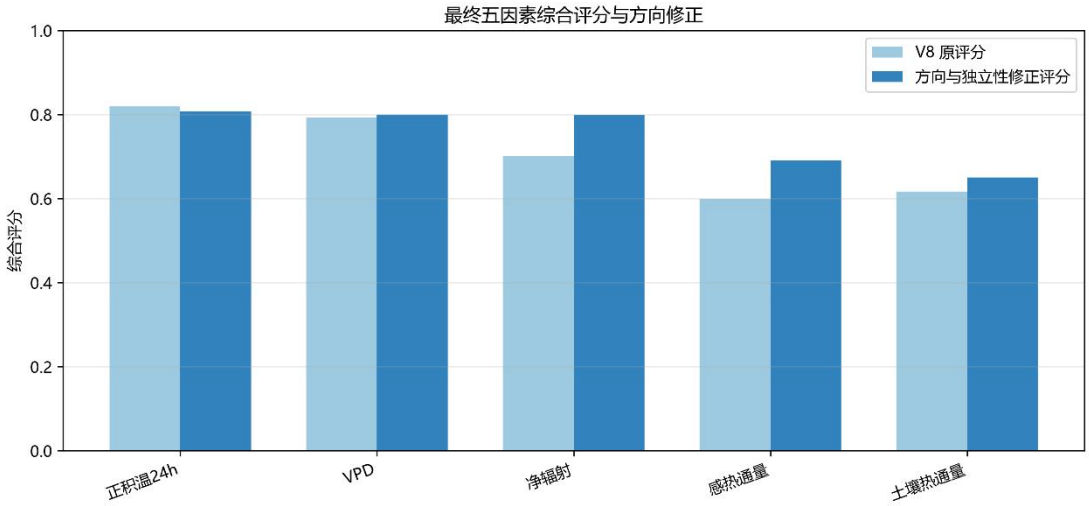


图 4-7 五因素综合评分与方向修正

五、问题二的建模与求解

5.1 数据来源与补充数据处理

5.1.1 历史泥石流空间数据

赛题提供了吉隆县 84 处历史泥石流灾害范围数据，其中降雨型 41 处（48.81%）、冰水混合型 25 处（29.76%）、冰雪融水型 16 处（19.05%）、冰湖溃决型 2 处（2.38%）。该数据反映了研究区历史泥石流的类型组成及空间分布特征。由于影像获取日期和解译日期不等同于灾害实际发生日期，因此本文主要将其用于灾害类型与空间条件分析，不直接作为逐日气象风险模型的监督训练样本。

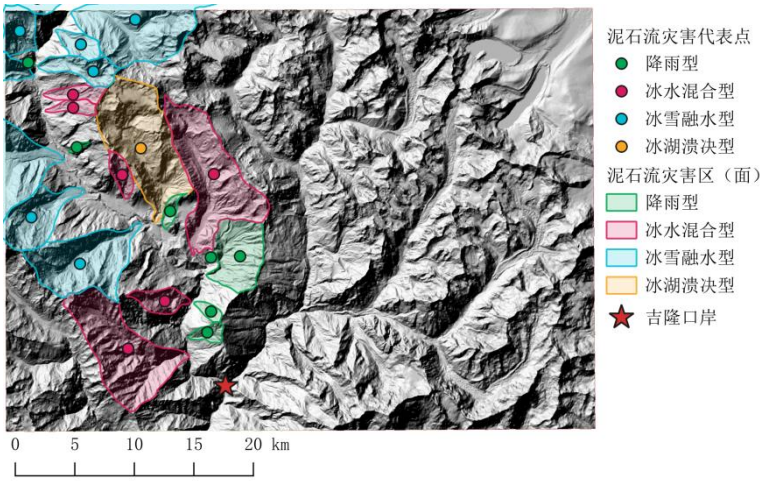


图 5-1 基于 DEM 底图的泥石流灾害空间样本分布图

上述数据能够反映历史泥石流的空间分布与类型差异，但其中影像获取日期、影像解译日期不能直接视为泥石流实际发生日期。因此，这 84 个灾害范围主要用于机制分析与空间条件研究，不直接用于构建逐日气象监督标签。

5.1.2 补充历史灾害日期数据

由于赛题历史灾害范围缺少足够明确的发生日期，难以与逐小时气象数据建立时间对应关系，本文另外收集并整理了 34 条带发生日期的吉隆县历史灾害记录，形成补充数据集《泥石流.csv》[4]。

该数据集用于弥补赛题空间资料在事件时间信息方面的不足，是本文动态触发模型的重要补充数据。整理结果如下。

34 条记录涉及 19 个发生日期，其中 2021 年 32 条记录对应 17 个独立事件日。本文以日期为时间分析单位，将同日多条记录合并，避免把共享同一气象过程的地点记录重复计作独立样本。

此外，自行补充的 34 条记录与赛题 84 个历史灾害范围并非已经确认的一一对应关系。本文不通过简单最近邻匹配强行确定其灾害类型，以避免错误标签传播。

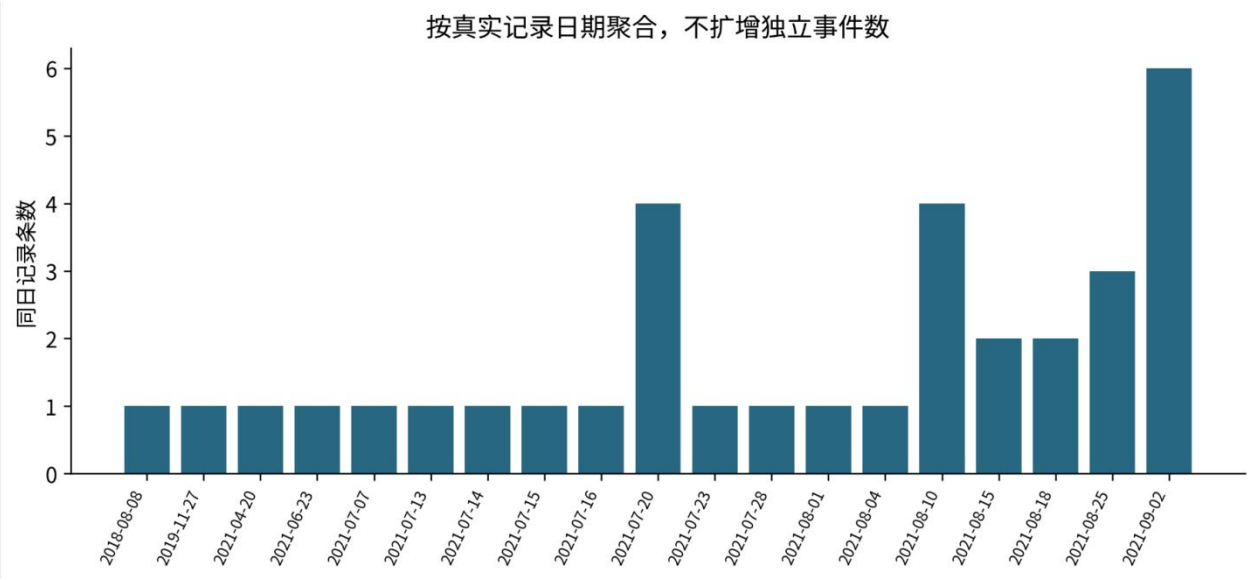


图 5-2 历史灾害台账真实发生日期分布统计图

5.1.3 气象观测数据

本文采用逐小时气象观测数据构建泥石流动态风险评价模型，选取降雨量和近地面气温作为主要输入变量，分别记为(P_t)和(T_t)，单位为 mm 和℃。首先按照时间戳对数据进行排序与对齐，将降雨量小于 0 或不小于 999mm、气温不大于−80℃或不小于 60℃的异常观测值设为缺失。为保证累计特征的完整性，仅使用观测数据完整的时间窗口，不对缺失值进行插值处理。以每日 00:00 为风险评估时刻，提取此前的历史气象信息，构建累计降雨量和正积温特征，为后续逻辑回归模型提供输入数据。

5.1.4 冰川数据

为分析冰雪相关泥石流的潜在源区条件，本文读取冰川 NetCDF 数据中的冰厚、冰流速度及坡度变量。各变量分别记为：

H：冰厚，单位 m；

V：冰流速度，单位 m/年；

θ：坡度，单位°。

冰川数据主要用于静态空间条件分析。由于缺少已确认的冰崩释放区和连续失稳过程观测，本文不将其直接作为 2021 年逐日气象模型的监督变量，也不利用单次 2026 年冰崩案例反向训练冰崩分类器。全文中统一采用P_{72}表示 72 小时累计降雨量，采用D_{72}表示 72 小时正积温。

5.2 动态气象触发模型建立

5.2.1 累计降雨指标

考虑降雨对泥石流形成的累积影响，分别构建 24h、72h 和 360h 累计降雨指标，以表征短期降水、三日降水及前期水分累积条件。定义：

$$P_n(t) = \sum_{k=1}^{n} P\,(t-k), \quad n \in \{24,72,360\} \tag{5-1}$$

其中，P(t − k)表示评估时刻之前第(k)个小时的降雨记录。

因此：

$$P_{24}(t) = \sum_{k=1}^{24} P(t-k) \tag{5-2}$$

$$P_{72}(t) = \sum_{k=1}^{72} P(t-k) \tag{5-3}$$

$$P_{360}(t) = \sum_{k=1}^{360} P(t-k) \tag{5-4}$$

上述指标分别反映灾害风险评估时刻前 1 天、3 天和 15 天的累计降水情况。计算时采用滚动求和构建历史降雨特征，仅使用评估时刻之前的观测数据，为后续泥石流风险评价模型提供输入变量。

5.2.2 正积温指标

考虑气温对冰雪消融及泥石流水热条件的影响，采用 72h 累计正积温表征灾前热力环境。其计算公式为：

$$D_{72}(t) = \frac{1}{24}\sum_{k=1}^{72} \max\,[T(t-k), 0] \tag{5-5}$$

式中，T_{t-k}为评估时刻(t)之前第(k)小时的气温，单位为℃；$\max\,(T_{t-k}, 0)$表示仅累计高于 0℃的温度部分；D_{72}为 72h 累计正积温，单位为℃·日。该指标用于反映灾前温度累积特征，并与累计降雨量共同构成泥石流风险评价模型的输入变量。

5.2.3 特征标准化

不同特征具有不同的数量级和单位。为避免较大数值尺度影响正则化逻辑回归的系数估计，对各特征进行标准化：

$$Z_{t,j} = \frac{X_{t,j} - \mu_j}{\sigma_j} \tag{5-6}$$

其中，μ_j和σ_j仅由训练集计算，测试集使用相同参数变换。这样能够避免测试期统计信息参与训练。

5.2.4 逻辑回归评分模型

设第 t 日的特征向量为：

$$Z_t = (Z_{t,1}, Z_{t,2}, ..., Z_{t,p})^T \tag{5-7}$$

定义线性评分：

$$\eta_t = \beta_0 + \sum_{j=1}^{p} \beta_j Z_{t,j} \tag{5-8}$$

通过 Sigmoid 函数得到：

$$S_t = \frac{1}{1+e^{-\eta_t}} \tag{5-9}$$

其中，s_t用于衡量当前气象条件与历史记录事件日特征的相似程度。

模型采用 L2 正则化逻辑回归，通过最小化带惩罚项的二元交叉熵估计参数：

$$\min_{\beta,\beta_0} \sum_{t\in T_{train}} w_{y_t}[\ln(1+e^{\eta_t}) - y_t\eta_t] + \frac{||\beta||^2}{2C} \tag{5-10}$$

其中：

T_{train}为训练日期集合；

w_{y_t}为类别平衡权重；

C = 0.1 为正则化参数；

截距不参与上述系数惩罚。

本文采用类别平衡处理，减轻记录事件日数量较少造成的类别不均衡影响。由于背景日期未经过完整无灾核实，且模型使用类别权重，输出s_t不直接解释为实际泥石流发生概率，而作为经验判别评分。

5.2.5 报警评分分界

为将逻辑回归输出的连续风险评分转化为预警决策，采用 ROC 曲线与 Youden 指数确定最优报警阈值。定义 Youden 指数为：

$$J(c) = TPR(c) - FPR(c) \tag{5-11}$$

最优阈值为：

$$\tau^* = \arg\max_c J(c) \tag{5-12}$$

其中，真正例率和假正例率分别定义为：

$$TPR = \frac{TP}{(TP+FN)} \tag{5-13}$$

$$FPR = \frac{FP}{(FP+TN)} \tag{5-14}$$

最终报警规则为：

$$\hat{y}_t = \begin{cases} 1, S_t \geq \tau^* \\ 0, S_t < \tau^* \end{cases} \tag{5-15}$$

其中，S_t为模型输出的风险评分，$\hat{y}_t$表示触发预警。报警阈值仅由训练集确定，并在测试阶段保持不变，以避免测试信息参与模型训练。

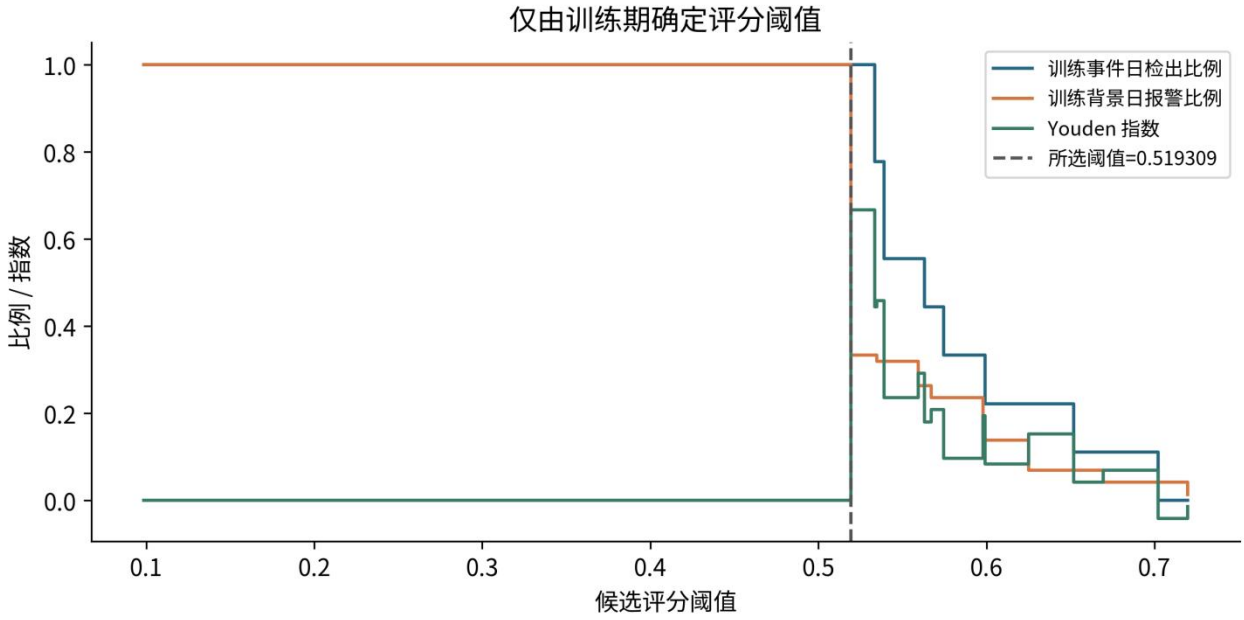


图 5-3 基于 Youden 指数的训练期最优评分阈值寻优图

5.3 模型求解与实验设计

5.3.1 时间划分

为评价模型在未来日期上的表现，本文采用时间前向划分，不随机打乱日期样本。具体划分如下。

表 3 训练集、隔离期与测试集划分

数据集	时间范围	有效日期	记录事件日
训练集	2021 年 4 月 1 日至 7 月 31 日内的有效日期	81	9
隔离期	2021 年 8 月 1 日至 8 月 15 日	不参与训练或测试	—
测试集	2021 年 8 月 16 日至 9 月 30 日内的有效日期	46	3

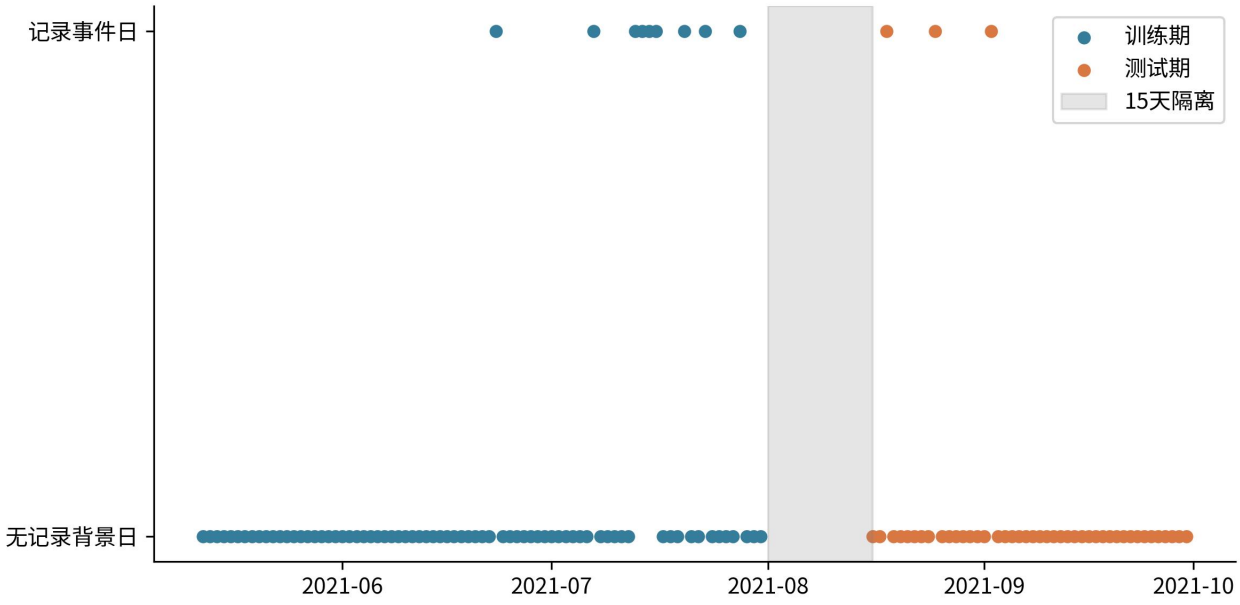


图 5-4 气象窗口防数据泄露的时间切分策略

训练与测试之间设置 15 天隔离期，减少 360 小时累计降雨窗口造成的跨边界信息重叠。

这里的 81 天与 46 天均为通过气象完整性检查后的有效样本日期，不代表对应时间段内所有自然日均有可用观测。

此外，测试集只有 3 个记录事件日，统计指标可能因个别事件而发生较大变化，因此评价结果主要用于探索性比较。

5.3.2 候选模型设置

为比较不同降雨累计窗口的作用，建立以下四种模型。

所有模型采用相同的训练日期、测试日期、标准化方法和逻辑回归正则化参数，保证结果具有可比性。

5.3.3 模型评价指标

使用 ROC 曲线下面积 AUC、平均精确率 AP、事件召回率和背景日报警比例评价模型。

事件召回率定义为：

$$R = TP/(TP + FN) \tag{5-16}$$

背景日预警比例定义为：

$$FPR_{bg} = \frac{FP}{FP+TN} \tag{5-17}$$

其中，背景日指没有对应灾害记录的日期，不等同于已经核实的无灾日期。AP 用于评价模型对少数记录事件日的排序能力，并与测试集事件日占比共同解释。

5.4 模型求解结果

5.4.1 不同降雨窗口模型比较

四种模型的测试结果如下。

表 4 不同特征组合的测试集表现

模型	AUC	AP	事件召回率	背景日报警比例
基准模型	0.5349	0.0890	66.67%	51.16%
P_{24}+D_{72}	0.6899	0.1328	66.67%	39.53%
P_{72}+D_{72}	0.7132	0.1546	66.67%	32.56%
P_{360}+D_{72}	0.4961	0.0826	33.33%	51.16%

在固定测试集上，P_{72}+D_{72}的 AUC 与 AP 数值最高；与基准模型相比，其在识别相同数量事件日的同时减少了背景日报警。由于候选模型依据同一测试集筛选，该排序仅作探索性比较，不代表独立泛化性能。

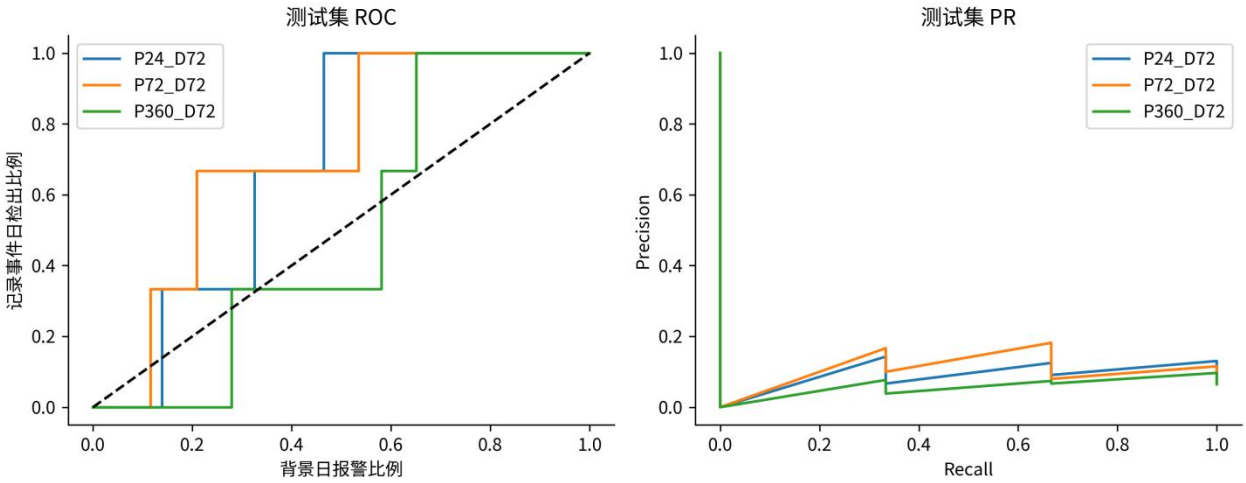


图 5-5 不同时间窗口特征组合的模型测试集 ROC 与 PR 曲线对比

5.4.2 P_{72}+D_{72}模型参数

P_{72}+D_{72}模型在训练集上得到的标准化参数为：

$$\beta_0 = -0.2777 \tag{5-18}$$

$$\beta_1 = 0.1998 \tag{5-19}$$

$$\beta_2 = 0.7153 \tag{5-20}$$

训练集标准化参数为：

$$\mu_P = 4.7901234568, \sigma_P = 7.8600663607 \tag{5-21}$$

$$\mu_D = 31.3607664609, \sigma_D = 7.1384357546 \tag{5-22}$$

因此，模型线性评分为：

$$\eta_t = -3.542046 + 0.025422P_{72}(t) + 0.100206D_{72}(t) \tag{5-23}$$

将标准化项展开，得到原始单位下的近似表达式：

$$s_t = \frac{1}{1+e^{(-\eta_t)}} \tag{5-24}$$

上述系数是根据实际训练结果换算得到的经验回归系数。

P_{72}的单位为 mm，D_{72}的单位为℃·日，两个回归系数具有不同量纲，不能仅凭系数数值大小比较两种物理因素的重要性。

5.4.3 经验联合临界条件

P_{72}+D_{72}模型在训练期确定的报警评分分界为：

$$s_c = 0.5193 \tag{5-25}$$

根据 Sigmoid 函数的反变换：

$$\eta_c = ln\frac{s_c}{1-s_c} \tag{5-26}$$

计算得到：

$$\eta_c \approx 0.077273 \tag{5-27}$$

结合模型原始单位表达式，得到：

$$0.025422P_{72} + 0.100206D_{72} \geq 3.619319 \tag{5-28}$$

整理可得：

$$0.025422P_{72} + 0.100206D_{72} \geq 3.619319 \tag{5-29}$$

该式为本文得到的 72 小时累计降雨—72 小时正积温经验联合筛查条件。

进一步整理，可得到给定正积温条件下的经验降雨分界：

$$P_{72,c}(D_{72}) = \frac{3.619319-0.100206D_{72}}{0.025422} \tag{5-30}$$

当实际 72 小时累计降雨量满足：

$$P_{72} \geq P_{72,c}(D_{72}) \tag{5-31}$$

时，模型评分达到训练期报警分界。

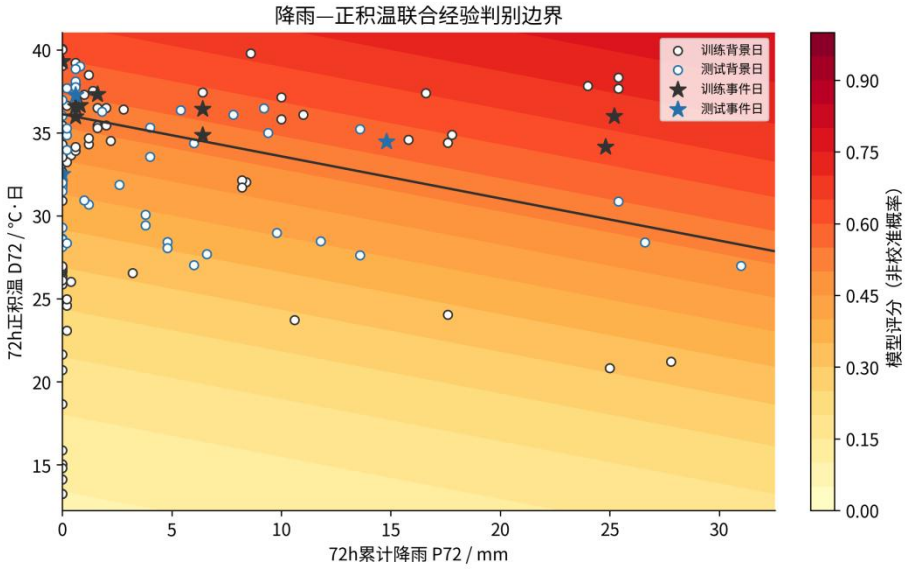


图 5-6 72 小时累积降雨与正积温的联合经验判别边界与风险分布图

当计算得到的临界降雨量为负值时，表明正积温指标已使模型评分达到报警阈值，此时无需额外降雨即可满足模型的经验判别条件。该判别边界仅适用于训练数据所覆盖的气象特征范围。

5.4.4 测试集判别结果

P_{72}+D_{72}模型在测试集上的混淆矩阵如下，行表示真实类别，列表示模型判定类别：

表 5　P72+D72 模型测试集混淆矩阵

真实类别	预测为事件	预测为背景
记录事件日	2	1
背景日	14	29

由此得到：

$$R = 2/3 = 66.67 \tag{5-32}$$

$$FPR_{bg} = 14/43 = 32.56 \tag{5-33}$$

测试集事件日占比为：

$$\pi = 3/46 = 6.52 \tag{5-34}$$

模型 AP 为 0.1546，高于测试集事件日占比 0.0652，表明模型具有一定的事件日识别能力。但受测试集正例数量较少（仅 3 天）的限制，其预测性能仍需进一步验证。

以 2026 年 8 月 26 日吉隆泥石流事件为例，对气象风险模型进行回溯分析。结果表明，基准模型风险评分为 0.6930，高于训练期确定的报警阈值 0.4901，说明该事件对应的气象条件满足模型的经验预警判别条件。该结果为模型的案例分析提供了参考。

5.5 冰川静态条件分析

5.5.1 冰川静态变量统计

为分析研究区冰川环境的空间特征，基于 NetCDF 数据提取冰川厚度、冰流速度及坡度三个指标，并对有效栅格像元进行统计，分析其数值分布与空间差异。三类指标的空间分布如图 5-1 所示，为潜在冰川灾害源区的辅助识别提供基础数据。

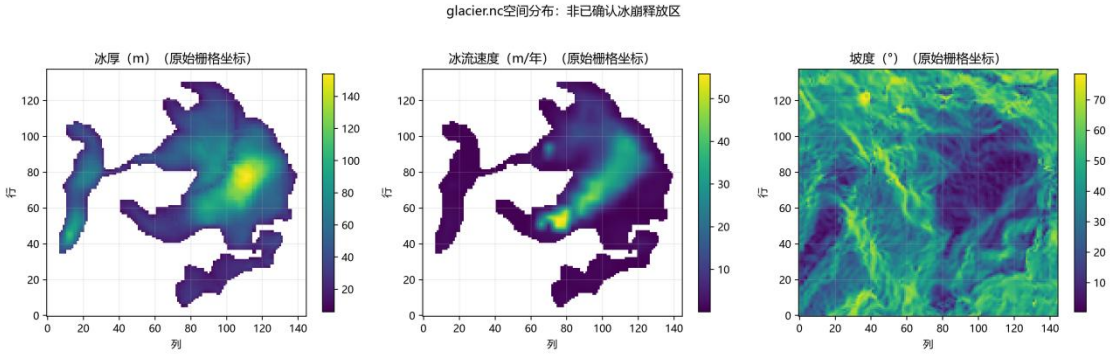


图 5-7 冰厚、冰流速度和坡度的原始栅格空间分布

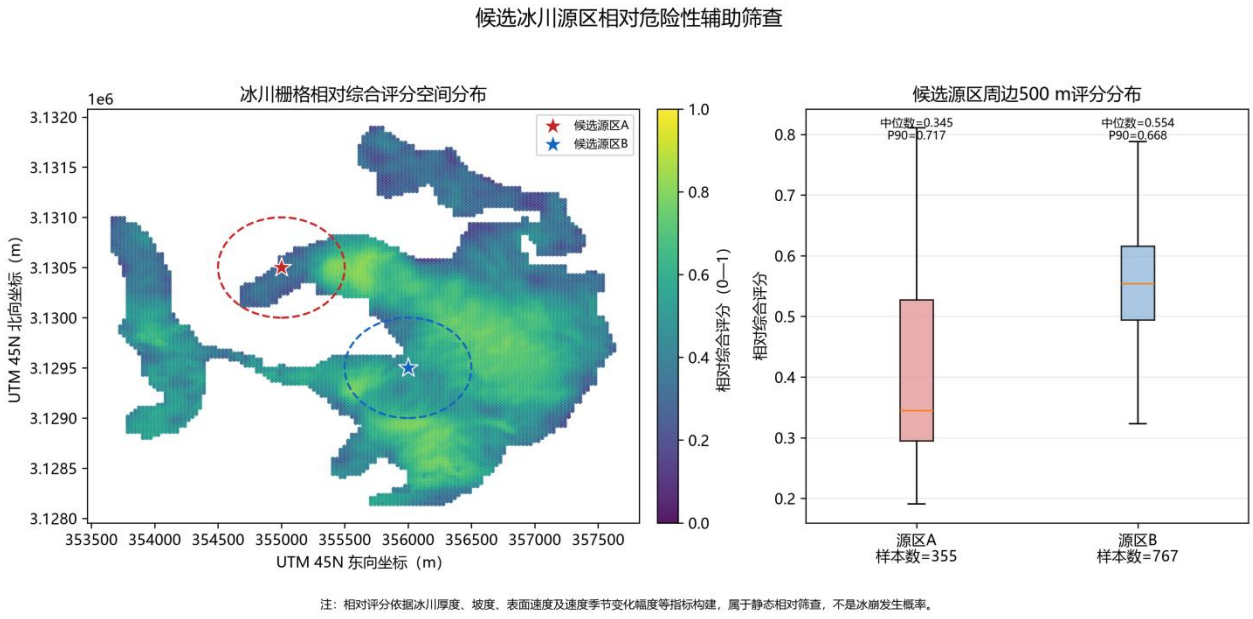


图 5-8[15]冰川静态相对危险性筛查

六、问题三的建模与求解

6.1 48 小时气象风险预警模型

6.1.1 气象风险因子

设R_t为时刻t的小时降水量，T_t为对应气温。72 小时累计降水量定义为

$$P_{72,t} = \sum_{k=0}^{71} R_{t-1-k} \tag{6-1}$$

72 小时正积温（单位为°C·天）定义为

$$D_{72,t} = \frac{1}{24}\sum_{k=0}^{71} \max(T_{t-k}, 0) \tag{6-2}$$

其中，72 小时累计降水量反映近期降雨条件，正积温用于表征近期正温融化的气象条件。

6.1.2 冻结的第二问逻辑回归模型

沿用第二问已经训练完成的模型：

$$S_t = \frac{1}{1+exp(-z_t)} \tag{6-3}$$

其中

$$z_t = -3.542046 + 0.025422P_{72,t} + 0.100206D_{72,t} \tag{6-4}$$

s_t表示气象风险评分。

模型最初按日期级事件标签训练。逐小时更新的s_t仅作为气象风险评分，不代表该小时泥石流触发概率。

6.1.3 未来 48 小时事件风险情景估计

为形成可操作的事前预警，将训练期校准得到的日风险情景参数记为p_t，并在风险状态持续假设下换算未来 48 小时至少出现一次记录事件的风险情景值。

设经过校准的单日风险参数为p_t，则采用连续时域危险率近似：

$$\lambda_t = -\frac{\ln(1-p_t)}{24}. \tag{6-5}$$

对于未来H小时：

$$q_t(H) = 1 - exp(-H * \lambda_t) \tag{6-6}$$

最终取$H = 48$，得到$q_t(48)$。

该换算依赖风险状态持续与常数危险率近似；所得$q_t(48)$是情景风险指标，不能视为独立验证过的真实 48 小时发生概率。

6.1.4 预警触发规则

通过训练期阈值搜索，最终确定 48 小时风险预警阈值：

$$q_c = 0.216337 \tag{6-7}$$

预警决策为：

$$A_t = \begin{cases} 1, & q_t(48) \geq 0.216337, \\ 0, & q_t(48) < 0.216337 \end{cases} \tag{6-8}$$

模型每日 00:00 发布一次未来 48 小时的风险预测。

这里的 0.216337 作用于 48 小时风险情景估计$q_t(48)$，不是原始逻辑回归评分s_t的阈值。

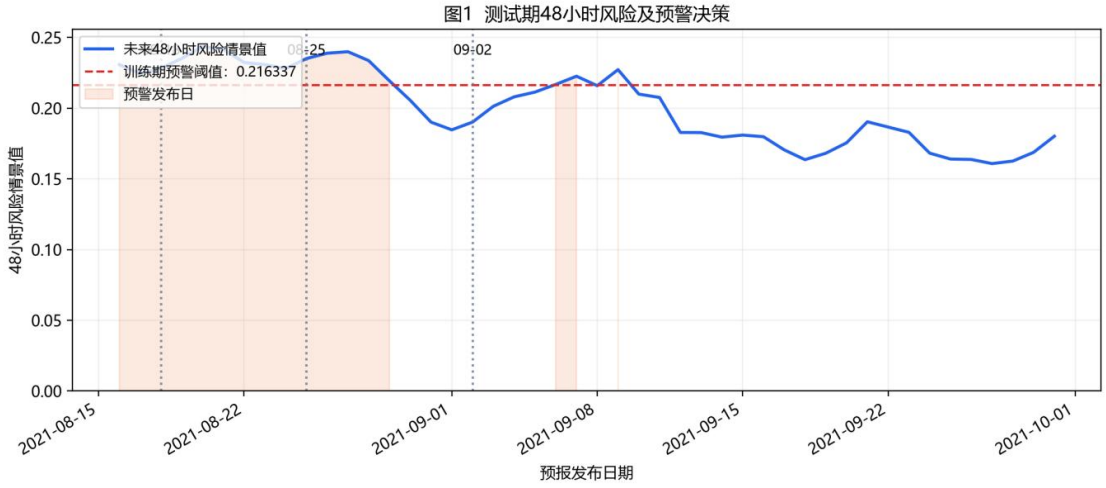


图 6-1 测试期 48 小时风险及预警决策

6.2 名义 95%触发时间预测区间

6.2.1 条件触发时间分布

设预报发布时间为t_0，预测时域为$H = 48$ 小时，事件触发时间距发布时间的时间差为τ。

在常数危险率λ假设下，若已知事件将在未来 48 小时内发生，其条件累积分布函数为

$$F(\tau|T \le H) = \frac{1-exp(-\lambda\tau)}{1-exp(-\lambda H)}, \quad 0 \le \tau \le H \tag{6-9}$$

由分位数函数

$$F^{-1}(u) = -\frac{1}{\lambda}\ln[1 - u(1 - e^{-\lambda H})] \tag{6-10}$$

构造名义 95%等尾条件时间区间：

$$I_{95} = [t_0 + F^{-1}(0.025), t_0 + F^{-1}(0.975)] \tag{6-11}$$

程序按小时离散计算，故区间端点均落在小时网格上。该区间表示：在事件确实发生于未来 48 小时这一条件下，模型分配约 95%条件概率质量的时间范围。

6.2.2 2026 年典型事件的预测结果

以 2026 年 8 月 25 日 10:00 为预警发布时间，模型计算得到未来 48h 风险情景值为 0.234617，高于报警阈值 0.216337，触发风险预警。对应的名义 95%条件时间预测区间为 8 月 25 日 11:00 至 8 月 27 日 09:00，区间宽度为 46h，覆盖 8 月 26 日 10:52 的冰岩崩塌起始参考时刻。预警发布时间距该参考时刻约 24h52min，表明模型在此次案例回溯分析中能够提前识别风险，并给出包含参考时刻的条件时间区间。

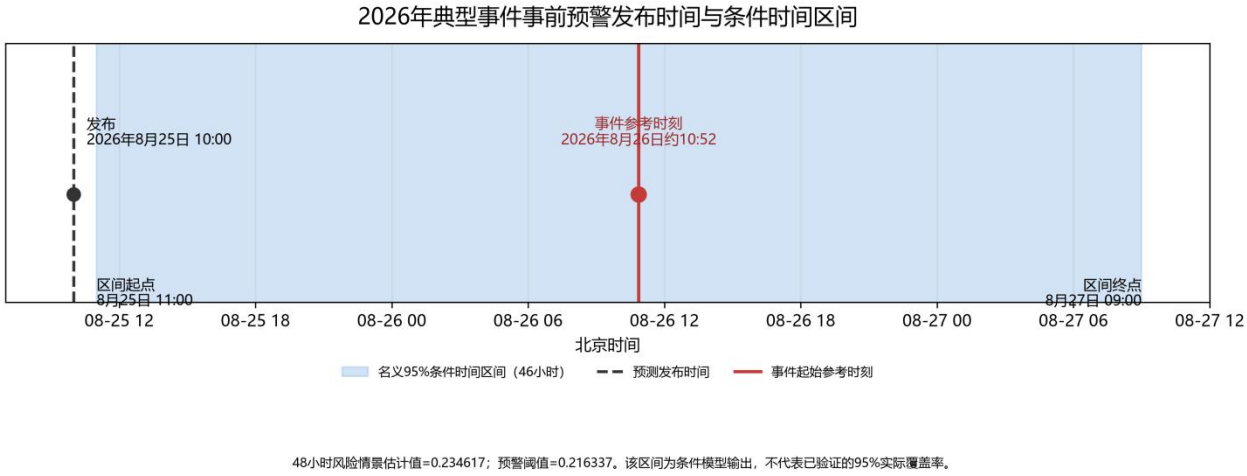


图 6-2 2026 年典型事件事前预警发布时间与 95%条件时间预测区间

6.3 基于区间删失的时间区间评价

由于历史泥石流记录仅包含发生日期，缺少具体发生小时，因此采用区间删失方法评价预测时间区间的覆盖情况。设历史灾害记录日期为(d_i)，则其实际发生时刻(T_i)满足：

$$T_i \in [d_i,\ d_i + 24\text{h})$$

其中，d_i表示记录日期的 00:00。根据真实发生时间区间与模型预测区间的重叠关系，评价模型对历史灾害事件的时间覆盖情况。

6.3.1 严格覆盖率上下界

将每个日期对应的 24 小时不确定区间与预测区间比较，按完全包含、完全不相交和部分重叠分类；据此计算严格覆盖率上下界。$I_i = [a_i, b_i)D_iI_i$

设独立事件日期总数为N，确定覆盖数为N_F，部分重叠数为N_P。则实际覆盖率的可判定界限为：

$$C_L = \frac{N_F}{N}, \qquad C_U = \frac{N_F + N_P}{N} \tag{6-14}$$

6.3.2 模糊加权覆盖率

为了充分利用日期级时间信息，在附加“事件当天发生时刻均匀分布”的假设下，定义单个事件的模糊覆盖权重：

$$w_i = \frac{|D_i \cap I_i|}{|D_i|} \tag{6-15}$$

由此计算：

$$C_{fuzzy} = \frac{1}{N} \sum_{i=1}^{N} w_i \tag{6-16}$$

该指标用于衡量预测区间与历史事件发生时段的平均重叠程度，为时间预测区间的评价提供补充依据。

6.3.3 模糊覆盖率计算结果

根据式（6-15）和式（6-16），计算得到历史事件日期窗口与预测区间的平均重叠时长约为 23h，对应的模糊加权覆盖率为：

$$C_{\text{fuzzy}} = \frac{23}{24} \times 100\% = 95.83\%$$

结果表明，预测区间与历史事件日期窗口具有较高的时间重叠程度。该指标基于事件发生时刻在当日均匀分布的假设，用于辅助评价模型的时间区间预测效果。

6.4 预警时间与提前量计算

6.4.1 预警决策模型

将未来 48 小时事件风险情景值与训练期阈值比较，形成每日预警状态。当连续多天满足预警条件时，将其合并为同一持续预警片段，避免将连续报警误计为多次独立预警。

设事件发生时刻为T_i，对应事前预报发布时间为t_i，则提前量为：

$$\Delta t_i = T_i - t_i. \tag{6-18}$$

若只知道事件发生日期：

$$T_i \in [L_i, U_i) \tag{6-19}$$

那么提前量满足：

$$\Delta t_i \in [L_i - t_i, U_i - t_i) \tag{6-20}$$

6.4.2 2021 年独立测试事件的预警结果

测试期 8 月 18 日和 8 月 25 日两个记录事件日被命中，9 月 2 日未命中；事件日期级检出率为：

$$R_{event} = \frac{2}{3} \times 100\% = 66.67\% \tag{6-21}$$

两次检出对应同一持续预警片段，其首次报警时间为 2021 年 8 月 16 日 00:00，表明模型能够识别持续性风险阶段，但尚不能对两次事件分别实现独立的短临预警。

6.5 泥石流空间传播路径分析

基于数字高程模型（DEM），采用 D8 算法确定地表最陡下降方向，并追踪候选源区的潜在汇流路径。计算结果表明，两个候选源区对应的路径长度分别为 26.24km 和 28.00km，路径终点距吉隆口岸参考点约 459m。结果反映了研究区地形条件下的潜在汇流方向与传播通道，为泥石流空间风险分析提供参考。

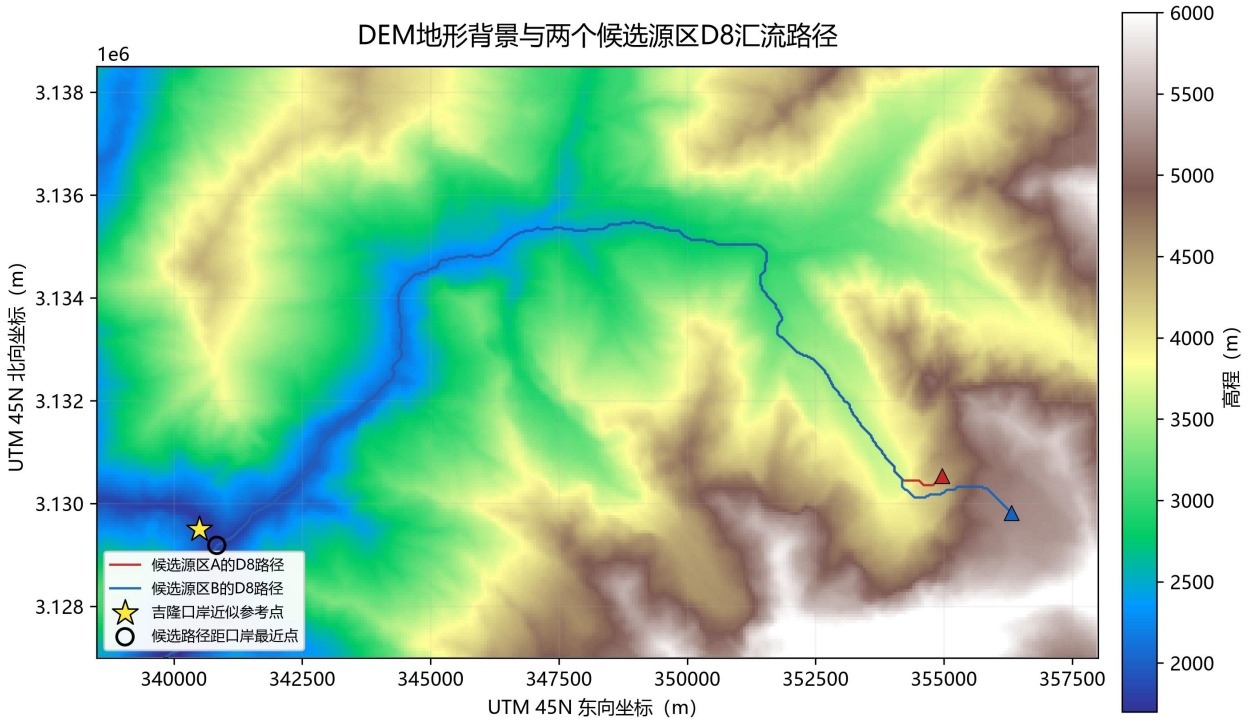


图 6-3 吉隆口岸周边地形背景与潜在泥石流传播路径

6.6 模型对比与最终选择

为确定最终预警模型，对比基础模型与动态特征增强模型的预测效果。结果表明，动态增强后，测试窗口 AUC 由 0.6792 下降至 0.6625，事件日检出率均为 66.67%，背景窗口报警比例由 32.50%上升至 45.00%。综合考虑模型预测性能、复杂度及可解释性，最终选用以 72h 累计降雨量和 72h 正积温为输入的 P72+D72 模型，构建 48h 泥石流风险预警体系。

七、第四问模型建立与求解

7.1 研究区地形与初始条件构建

7.1.1 地形数据及坐标统一

本问以吉隆“8·26”冰岩崩—泥石流灾害链为研究对象，利用附件 DEM 建立二维空间计算区域。原始地形数据经投影转换至 UTM45N 坐标系（EPSG:32645），统一采用米作为水平坐标和高程单位。DEM 原始分辨率为 30m，数值模拟采用约 90m 计算网格；三维展示所用的网格采样与数值计算网格分开处理。

设床面高程为$z(x,y,t)$，初始地形为$z_{\mathrm{o}}(x,y)$，其高程梯度控制重力驱动方向：

$$\mathrm{S_x} = -\frac{\partial \mathrm{z}}{\partial \mathrm{x}}, \qquad \mathrm{S_y} = -\frac{\partial \mathrm{z}}{\partial \mathrm{y}} \tag{7-1}$$

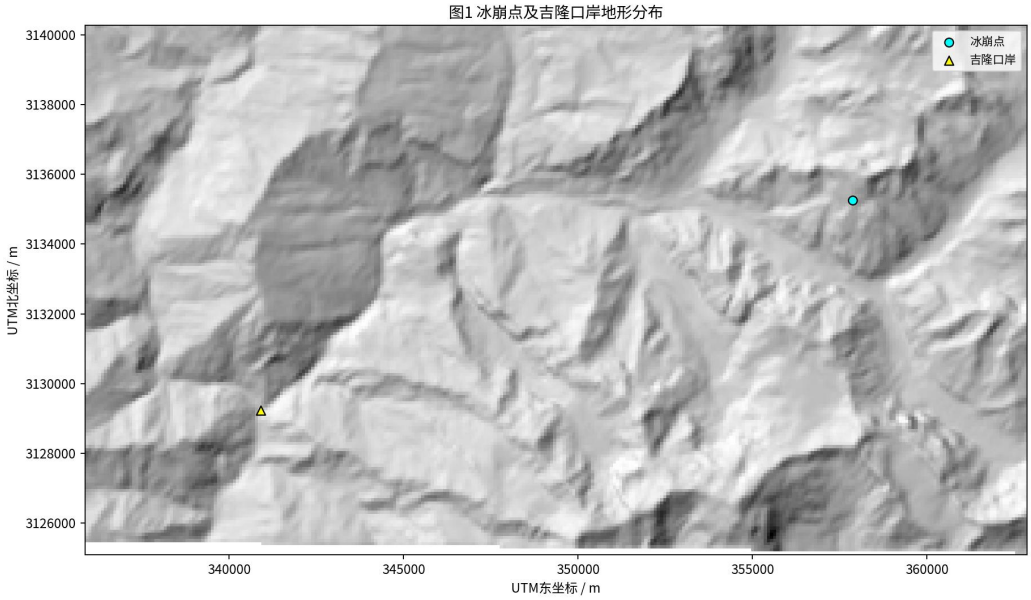


图 7-1 显示冰崩点与吉隆口岸在研究区内的相对位置

地形起伏及沟谷走向为后续二维计算域和传播路径的确定提供空间依据。

7.1.2 初始释放区域和物源厚度

针对文献中的源区识别结果存在差别的问题，主情景[6]给出的初始物源规模约 $1.75\times10^{8}\,\mathrm{m^3}$，对应约 $4.0\times10^{6}\,\mathrm{m^2}$的释放面积；实际栅格化源区面积约 $4.026\times10^{6}\,\mathrm{m^2}$。将初始物源体积等效分配到所构建的释放区域，平均初始厚度约 43.47m。所用释放区几何是以文献定位为依据的数值近似，并非实测源区多边形。

设释放区为Ω_{o}，初始体积为V_{o}，面积为A_{o}。初始流动层厚度表示为：

$$\mathrm{h(x,y,0)} = \begin{cases} \frac{\mathrm{V_0}}{\mathrm{A_0}}, & (\mathrm{x,y}) \in \Omega_0, \\ 0, & (\mathrm{x,y}) \notin \Omega_0. \end{cases} \tag{7-2}$$

因此，初始平均厚度的计算为：

$$\bar{h}_0 = \frac{V_0}{A_0} \approx \frac{1.75\times10^8}{4.026\times10^6} = 43.47\ \text{m}. \tag{7-3}$$

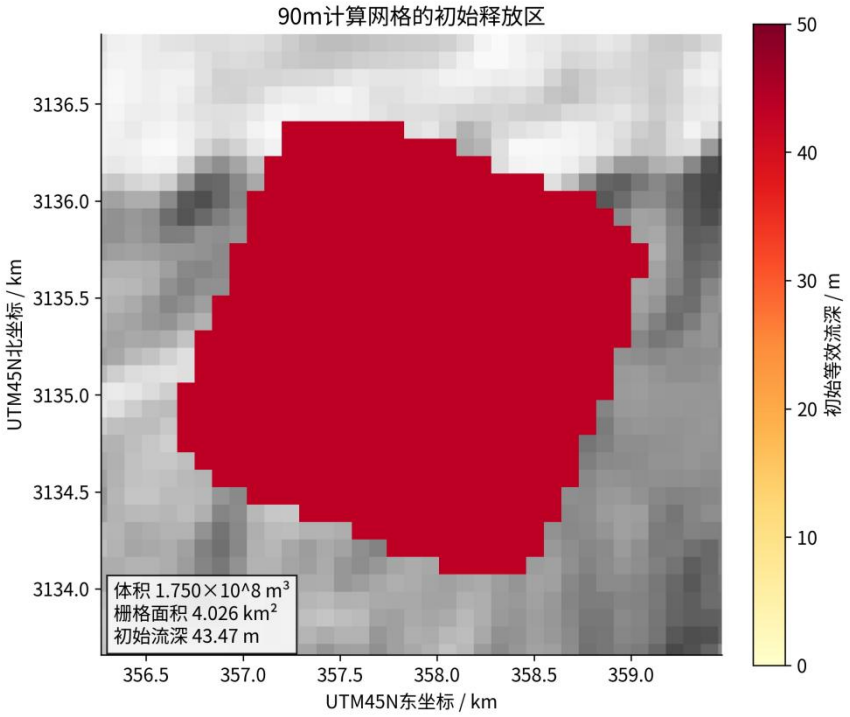


图 7-2 初始释放区域及其等效流层厚度

图 7-2 给出初始释放区域及其等效流层厚度。释放区[5]由文献定位信息与栅格化计算近似构造，故其空间边界和平均厚度均受源区解释误差影响。

7.2 二维深度平均动力学模型

7.2.1 基本假设与守恒变量

灾害物质主要沿山谷向下游扩展，其水平方向传播尺度大于局部流层厚度，因此采用二维深度平均近似。将高浓度冰岩碎屑—泥石流混合物视为等效连续介质，在每个网格中以流层厚度和两个水平动量表示其运动状态。模型不单独求解冰、水和颗粒各组分的速度场。

定义流层厚度$h(x,y,t)$、深度平均速度$u(x,y,t), v(x,y,t)$和守恒变量向量：

$$\mathbf{U} = (h, hu, hv)^{\mathrm{T}} \tag{7-4}$$

设E和D分别为单位面积、单位时间内的等效侵蚀率与沉积率，则深度平均质量守恒方程为：

$$\frac{\partial h}{\partial t} + \frac{\partial (hu)}{\partial x} + \frac{\partial (hv)}{\partial y} = E - D \tag{7-5}$$

式（7-5）表示流动层体积受水平通量、床面侵蚀补给和低速沉积损失共同控制。

7.2.2 水平动量守恒

采用静水压力项近似，将两个水平坐标方向的动量方程合写为向量形式，令$\mathbf{u} = (u, v)$，则：

$$\frac{\partial(\mathrm{hu})}{\partial \mathrm{t}} + \nabla \cdot \left(\mathrm{hu} \otimes \mathrm{u} + \frac{\mathrm{gh}^2}{2}\mathrm{I}\right) = -\,\mathrm{gh}\nabla \mathrm{z} + \mathrm{S_f} + \mathrm{M} \tag{7-6}$$

其中，g为重力加速度，S_{fx}, S_{fy}为阻力源项，M_x, M_y表示侵蚀沉积过程的动量交换。程序中侵蚀进入的物质初始水平速度近似为零；沉积携带当地平均动量退出流动层，因此流深变化与动量更新应保持一致。

7.2.3 Voellmy 摩擦阻力

Voellmy[9]阻力考虑基底库仑摩擦和与速度平方有关的阻力。设速度大小为 $\mathrm{s} = \sqrt{\mathrm{u}^2 + \mathrm{v}^2}$，坡面局部倾角为$\theta$，则阻力剪应力为：

$$\tau_\mathrm{f} = \mu\rho\mathrm{ghcos}\theta + \frac{\rho \mathrm{gs}^2}{\xi} \tag{7-7}$$

其中μ为无量纲库仑摩擦系数，ρ为等效密度，ξ为速度阻力参数，按当前代码约定单位为 $\mathrm{m/s^2}$。用单位面积流层质量ρh除以式（7-7），得到阻力减速度大小：

$$a_f = \mu g \cos\theta + \frac{gs^2}{\xi \max(h, h_{\min})} \tag{7-8}$$

将阻力沿速度反方向投影到两个水平方向，可得：

$$\mathrm{S_f} = -\,\mathrm{ha_f}\frac{\mathrm{u}}{\max(\mathrm{s}, \mathrm{s_{min}})} \tag{7-9}$$

其中$h_{\min}$和$s_{\min}$是防止干湿边界数值除零的正则化阈值；数值实现还限制单步摩擦不能令速度反向。考虑前期高速冰岩碎屑流和后期泥石流的等效阻力差异，采用分阶段摩擦系数：

$$\mu(t) = \begin{cases} \mu_0, & 0 \le t < t_c, \\ \mu_1, & t \ge t_c \end{cases} \tag{7-10}$$

当前代码取$t_c = 100$ s，这是简化的参数切换设置；μ_0, μ_1, ξ均属于该数值模型下试算得到的候选参数。

7.3 侵蚀、沉积与床面变化耦合

7.3.1 速度阈值侵蚀模型

冰岩碎屑流在高速沿沟运动过程中，可铲刮并夹带沟床及两岸松散物质。为描述这一过程，在质量守恒方程中引入经验侵蚀源项：

$$E = k\max(s - s_c, 0)\min\left(1, \frac{h}{h_c}\right) \tag{7-11}$$

式中k是与本经验公式绑定的无量纲侵蚀系数，s_c为临界侵蚀速度，h_c为薄流层侵蚀修正尺度。若流速未达到阈值，则侵蚀率为零。为了防止床层被无限侵蚀，在每个网格施加可侵蚀厚度约束：

$$0 \le e_{\mathrm{cum}}(x,y,t) \le H_e \tag{7-12}$$

7.3.2 沉积过程和床面更新

在低流速区域，流动层的一部分物质逐渐沉积。当前模型采用：

$$\mathrm{D} = \begin{cases} \mathrm{k_d h}, & \mathrm{s} < \mathrm{s_d}, \\ 0, & \mathrm{s} \ge \mathrm{s_d}, \end{cases} \tag{7-13}$$

其中k_d是沉积速率系数，s_d为低速沉积阈值。按流层与床面之间的等效体积交换，床面高程演化可表示为：

$$\frac{\partial z}{\partial t} = -E + D. \tag{7-14}$$

至此，质量守恒、动量守恒、Voellmy 阻力、侵蚀沉积与床面更新组成统一的耦合动力学模型。

7.4 参数确定及数值计算方法

7.4.1 文献约束与参数筛选

为确定泥石流动力学模型参数，参考文献[7]提供的灾害运动特征，将上游传播时间约 130s、下游传播时间约 320s、总传播时间约 450s，以及两阶段平均速度 63.6m/s 和 46m/s 作为参数筛选的参考依据。同时，结合文献[8]报告的侵蚀规模，对模型侵蚀参数进行量级约束。通过多组参数试算，筛选与文献重建运动特征相近的参数组合，为后续数值模拟提供基础。

定义待筛选参数向量：

$$\boldsymbol{\vartheta} = (\mu_0, \mu_1, \xi, k) \tag{7-15}$$

若观测统计口径统一，可采用如下归一化误差构造多指标目标函数：

$$\mathrm{J}(\vartheta) = \mathrm{w_t}\left(\frac{\mathrm{T_{sim}} - \mathrm{T_{ref}}}{\mathrm{T_{ref}}}\right)^2 + \mathrm{w_e}\left(\frac{\mathrm{V_{e,sim}} - \mathrm{V_{e,ref}}}{\mathrm{V_{e,ref}}}\right)^2 + \mathrm{w_v}\left(\frac{\bar{\mathrm{s}}_{\mathrm{sim}} - \bar{\mathrm{s}}_{\mathrm{ref}}}{\bar{\mathrm{s}}_{\mathrm{ref}}}\right)^2 \tag{7-16}$$

采用组合试算与局部细化筛选候选参数；现有计算不能证明已获得全局最优解。

表 7　二维泥石流动力学模型主要参数与计算设置

参数	符号	当前取值
重力加速度	g	9.81m/s²
初始物源体积	V_0	1.75×10^8m³
初始释放区面积	A_0	约 4.026×10^6m²
初始平均流层厚度	$\bar{h}_0$	43.47m
前期摩擦系数	μ_0	0.060
后期摩擦系数	μ_1	0.010
Voellmy 阻力参数	ξ	5000m/s²
经验侵蚀系数	k	0.000678
摩擦阶段切换时间	t_c	100s
侵蚀速度阈值	s_c	15m/s
参考侵蚀流深	h_c	2m
可侵蚀床层厚度上限	H_e	25m
沉积速率系数	k_d	$0.0003s^{-1}$
低速沉积阈值	s_d	5m/s
DEM 原始空间分辨率	—	30m
数值计算网格	$\Delta x, \Delta y$	约 90m
CFL 系数	C_{CFL}	0.25
仿真时长	T	600s
保存帧间隔	Δt_{out}	10s

7.4.2 有限体积数值离散

将控制方程[11][13]写为守恒形式：

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} + \frac{\partial \mathbf{G}(\mathbf{U})}{\partial y} = \mathbf{S}(\mathbf{U}, z). \tag{7-17}$$

采用有限体积法对网格单元积分，得到一阶离散格式：

$$\mathbf{U}_{i,j}^{n+1} = \mathbf{U}_{i,j}^{n} - \frac{\Delta t}{\Delta x}\Delta\hat{\mathbf{F}}_{i,j} - \frac{\Delta t}{\Delta y}\Delta\hat{\mathbf{G}}_{i,j} + \Delta t\,\mathbf{S}_{i,j}. \tag{7-18}$$

其中$\Delta\hat{\mathbf{F}}_{i,j} = \hat{\mathbf{F}}_{i+1/2,j} - \hat{\mathbf{F}}_{i-1/2,j}$，$\Delta\hat{\mathbf{G}}_{i,j} = \hat{\mathbf{G}}_{i,j+1/2} - \hat{\mathbf{G}}_{i,j-1/2}$。数值通量采用 Rusanov 格式，地

形通量处采用静水重构处理湿干边界和高程梯度，之后通过分裂步实施 Voellmy 摩擦耗散及侵蚀沉积更新。时间步长满足：

$$\Delta t \le \frac{C_{\mathrm{CFL}}}{\max\limits_{i,j}\left[\frac{|u|+\sqrt{g\bar{h}}}{\Delta x}+\frac{|v|+\sqrt{g\bar{h}}}{\Delta y}\right]}. \tag{7-19}$$

实际程序另对单步时间增量设置上限，并在每个保存时刻输出流深场、速度场、累计侵蚀与沉积量。

7.5 泥石流运动过程与三维展示

7.5.1 二维时空演化

将计算结果表示为$h(x,y,t)$、$u(x,y,t)$、$v(x,y,t)$，流速模为：

$$s(x,y,t)=\sqrt{u^2(x,y,t)+v^2(x,y,t)}. \tag{7-20}$$

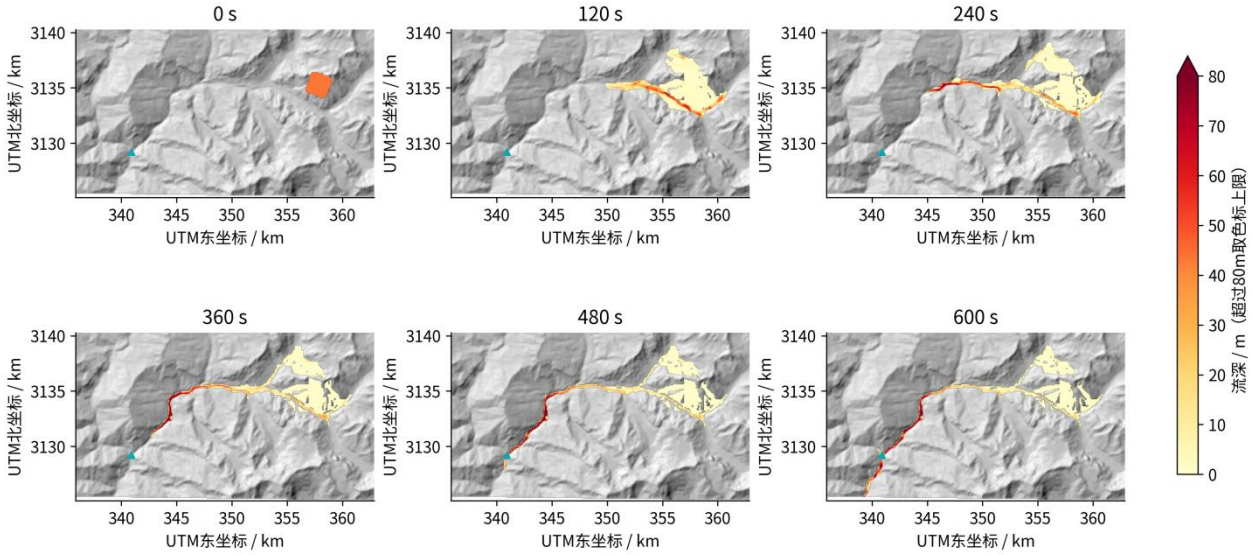


图 7-3 泥石流灾害链动力学演化过程多时相仿真图

7.5.2 三维地形动态展示

为展示模型模拟的三维空间形态，将二维计算所得流深场叠加于 DEM 地形表面，并对比 0、240 和 480s 三个时刻。图 7-3 依次呈现初始释放、沿沟谷下泄和向下游扩展的过程。该图用于辅助展示模型输出，不构成独立的实测验证。

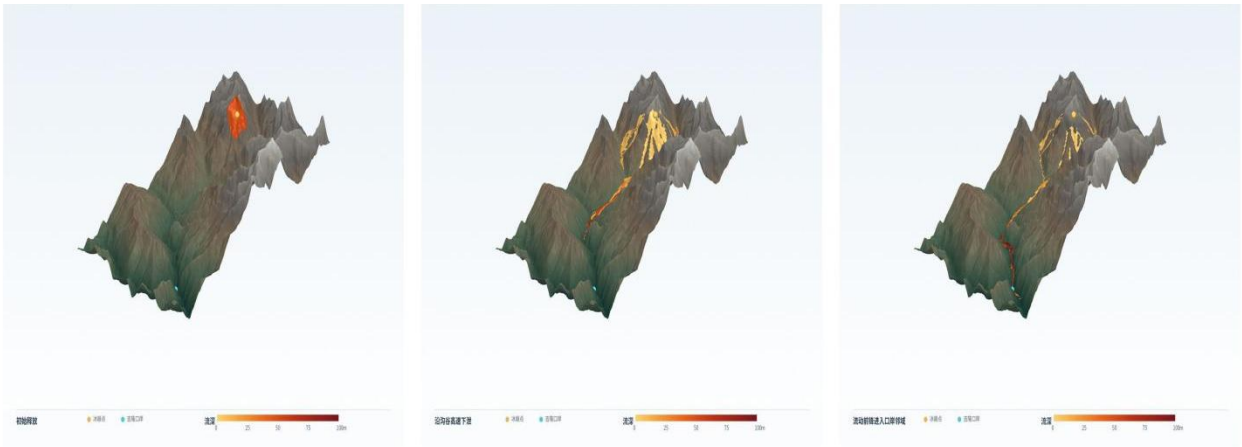


图 7-3 基于真实地形的泥石流三维运动演化过程（a）初始释放；（b）沿沟谷下泄；（c）前锋进入下游区域

7.6 运动强度及侵蚀沉积结果

7.6.1 峰值流深与流速

为反映计算期间各栅格的极值，定义历史最大流深和历史最大速度：

$$h_{\max}(x,y)=\max_{0\le t\le T}h(x,y,t),\qquad s_{\max}(x,y)=\max_{0\le t\le T}s(x,y,t). \tag{7-21}$$

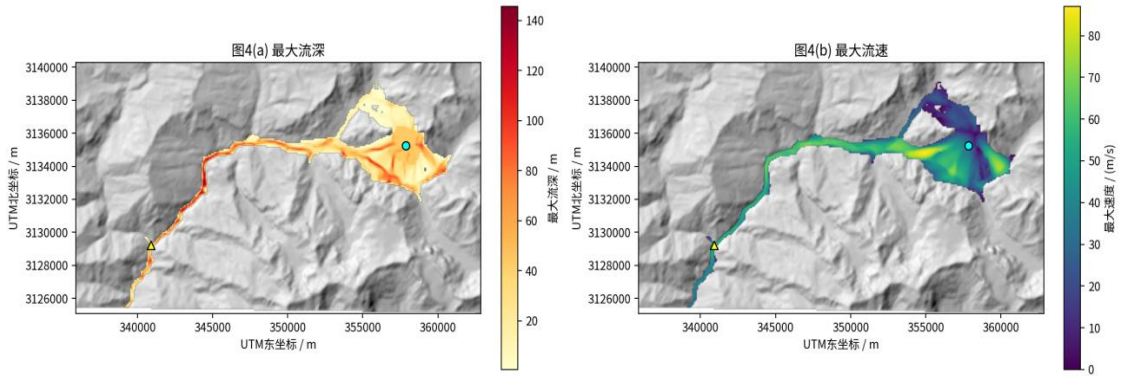


图 7-4 全时段最大流深与最大流速空间分布

7.6.2 侵蚀沉积空间分布

通过对侵蚀率和沉积率进行时间积分，得到累计侵蚀深度与累计沉积深度：

$$e_{\mathrm{cum}}(x,y,T)=\int_0^T E\,(x,y,t)\,dt,\qquad d_{\mathrm{cum}}(x,y,T)=\int_0^T D\,(x,y,t)\,dt. \tag{7-22}$$

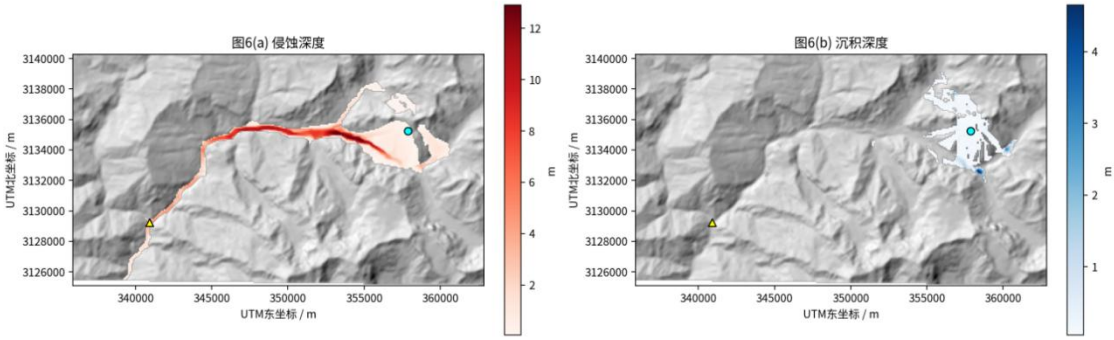


图 7-5 累计侵蚀与沉积空间分布

在 90m 计算网格、600s 模拟条件下，泥石流前锋于 440s 进入吉隆口岸 300m 邻域，最大格网流速为 86.99m/s，累计最大直线扩展距离为 21.106km，累计影响面积为 31.560km²。模拟域内累计侵蚀体积为 $5.8520\times10^{7}m^{3}$，累计沉积体积为 $3.3380\times10^{6}m^{3}$，600s 末流动层体积为 $2.3018\times10^{8}m^{3}$。模拟传播时间与文献重建的约 450s 在量级上接近，表明模型能够再现该参数情景下泥石流沿沟谷快速传播及侵蚀沉积的主要演化特征。

八、模型评价与讨论

本文围绕吉隆泥石流灾害的形成与演化过程，建立了涵盖灾前环境异常识别、触发风险评价、时空动态预警及泥石流动力学模拟的多层次建模体系。模型充分利用气象、辐射、土壤、冰川及 DEM 等多源数据，实现了统计分析与物理机制建模的结合，具有较好的系统性、可解释性和应用参考价值。在模型评价方面，问题一通过历史同期与暖季背景对照识别关键环境因素，增强了异常分析的客观性；问题二采用时间前向验证与 15 天隔离期，减少了模型训练与测试之间的信息重叠，P72+D72 模型测试 AUC 达到 0.7132；问题三构建 48h 风险预警与名义 95%条件时间预测区间，测试期事件日检出率为 66.67%，模糊加权覆盖率为 95.83%，实现了风险识别与时间范围估计的结合；问题四采用 Voellmy 摩擦阻力与侵蚀沉积耦合模型，模拟泥石流沿复杂地形的运动演化，600s 末相对体积收支残差约为-2.97×10^{-9}，表明数值计算具有较好的体积守恒表现。

模型仍存在一定局限性。首先，历史灾害记录较少，且部分记录缺乏准确发生时刻，影响统计模型的独立验证及预警时间精度；其次，单站气象数据与冰川源区之间存在空间代表性差异，模型对其他区域的适用性尚需检验。此外，时间预测区间的模糊覆盖率依赖日内均匀分布假设，不能直接等同于实际小时级覆盖率。动力学模拟结果对网格分辨率及经验参数具有一定敏感性，60m 与 90m 网格计算的传播时间分别约为 400s 和 440s，表明模拟精度仍有改进空间。后续可通过补充历史灾害样本、融合多站点观测数据、优化参数反演及开展多分辨率模拟，进一步提高模型的稳定性与应用可靠性。

九、参考文献

[1] 崔鹏.我国泥石流防治进展[J].中国水土保持科学,2009,7(5):7-13+31.DOI:10.16843/j.sswc.2009.05.002.

[2] 刘传正.中国崩塌滑坡泥石流灾害成因类型[J].地质论评,2014,60(4):858-868.DOI:10.16509/j.georeview.2014.04.017.

[3] 常康泰,赵志芳,牟乔木,等.基于灰色关联分析-信息量法的永胜县泥石流易发性评价[J].科学技术与工程,2025,25(3):933-941.

[4] 田建. 2022 年度西藏地质灾害普查数据[DS/OL]. V1. Science Data Bank, 2025[2026-10-30]. https://cstr.cn/31253.11.sciencedb.27813. CSTR:31253.11.sciencedb.27813.

[5] 中国岩石力学与工程学会. 中尼边境“8·26”冰岩崩—泥石流灾害链事件初步研判[EB/OL]. (2026-08-31)[2026-10-30]. https://info.csrme.com/Subsite/Index/show/id/5999.do.

[6] 刘昌军，等. 西藏吉隆“8·26”冰岩崩泥石流灾害链数字孪生推演分析[J]. 水利学报，2026，57(9). DOI: 10.3724/j.slxb.20260653.

[7] 杨云鹏，等. 基于地震动监测的“8·26”西藏吉隆泥石流灾害链初步解析[J]. 山地学报，2026，44(4): 525–533. DOI: 10.16089/j.cnki.1008-2786.000980.

[8] 殷跃平，等. 尼泊尔蓝塘里壤峰“8·26”冰岩崩致吉隆泥石流灾难研究[J]. 中国地质灾害与防治学报，2026. DOI: 10.16031/j.cghc.202610008.

[9] 胡雨昂,高杨,张晗,等.云南头寨高位远程崩滑-碎屑流多相态过程数值模拟[J/OL].地质力学学报,1-24[2026-10-30].https://link.cnki.net/urlid/11.3672.P.20261013.1500.002.

[10] 陈瑜彬,王正华,田逸飞,等.西藏吉隆泥石流灾害堰塞湖应急处置水文预报实践[J].水利水电快报,2026,47(10):1-7.DOI:10.15974/j.cnki.slsdkb.2026.10.001.

[11] 薛翊国,渠敬凯,彭建兵,等.雅鲁藏布江下游主要不良地质条件与重大工程地质问题[J].现代地质,2026,40(1):1-30.DOI:10.19657/j.geoscience.1000-8527.202511090148.

[12] Caine, N. (1980). The Rainfall Intensity - Duration Control of Shallow Landslides and Debris Flows. Geografiska Annaler: Series A, Physical Geography, 62(1–2), 23–27. https://doi.org/10.1080/04353676.1980.11879996

[13] LeVeque RJ. Finite Volume Methods for Hyperbolic Problems. Cambridge University Press; 2002.

[14] DeepSeek, DeepSeek-V4-Flash, 深度求索（DeepSeek）, [10.24]

[15] ChatGPT, GPT-5, OpenAI, [10.26]

十、附录

1.软件使用情况

序号	软件名称	版本或型号	使用方式	备注
1	DeepSeek	使用期间配置模型为 DeepSeek- V4- Flash	网页端	用于赛题理解、代码调试与文档整理
2	ChatGPT	使用期间配置模型为 GPT- 5	网页端	用于语言润色、代码辅助与图表建议
3	Pycharm	使用期间配置模型为 Pycharm2026.2.3	本地桌面	Python 代码编写、调试运行，时序风险计算、绘图脚本开发
4	QGIS	使用期间配置模型为 QGIS Desktop 3.44.15	本地桌面	DEM 预处理、填洼、D8 流向提取、汇流路径分析、冰川源区空间可视化

2.命令内容

2.1 使用记录 A01：赛题理解辅助

使用工具：DeepSeek

使用目的：将赛题逐句翻译为建模语言，识别隐藏条件与输入输出约束。

提供给 AI 的材料：赛题 PDF、已收集的参考资料。

主要提示方式：“请逐句翻译这道赛题，并梳理小问关系、输入输出与隐藏条件”。

使用过程：AI 输出题意逐句翻译、小问关系与约束清单；参赛队逐条对照原题复核后用于后续建模。

AI 输出概述：题意翻译报告、隐藏条件清单、小问关系流程图。

对应位置：论文“一、问题重述”“二、问题分析”。

2.2 使用记录 A02：数值求解辅助

使用工具：ChatGPT

使用目的：根据已确定的数学模型编写可运行代码，完成数值求解并输出结果附件。

提供给 AI 的材料：数学模型报告、赛题数据、已有思路说明。

主要提示方式：“请按这份数学模型写 Python 求解代码”“请帮我检查这段代码的约束是否写对”。

使用过程：AI 生成求解程序，本地运行并输出 result4- 3.xlsx。运行报错时修改后重跑。

AI 输出概述：Python 求解代码、各问题数值结果、结果附件。

对应位置：论文附录程序代码、result4- 3.xlsx。

2.3 使用记录 A03：代码调试辅助

使用工具：ChatGPT

使用目的：对运行报错进行定位，提出修改建议。

提供给 AI 的材料：报错信息、相关代码片段、数据结构说明。

主要提示方式：“运行报错信息如下，请帮我定位原因并给出修改建议”。

使用过程：AI 给出可能原因与修改方案；参赛队人工检查后修改代码并重跑验证。

AI 输出概述：错误原因分析、修改建议、代码片段。

对应位置：论文附录程序代码。

2.4 使用记录 A04：图表与流程图生成

使用工具：ChatGPT

使用目的：根据计算结果绘制论文所需的轨迹图、时序图、流程图和分配图。

提供给 AI 的材料：求解结果、代码、绘图要求。

主要提示方式：“请用 matplotlib 按这份数据画出时序图，并标注坐标轴与单位”。

使用过程：先由 AI 生成 matplotlib 图表，再由参赛队核对坐标轴、单位、图例与图注编号。

AI 输出概述：若干张论文图表。

对应位置：论文图 1~图 13。

2.5 使用记录 A05：语言润色辅助

使用工具：DeepSeek

使用目的：对论文摘要、章节文字进行语言润色与格式检查。

提供给 AI 的材料：论文草稿段落。

主要提示方式：“请在不改变原意的前提下润色这段文字，并检查格式”。

使用过程：AI 输出润色后的文字；参赛队逐句核对，确保不改变原意、不引入错误结论。

AI 输出概述：润色后的文字段落。

对应位置：论文全文。

3. 源程序

#Q2

```
    from pathlib import Path
import json
import warnings
import numpy as np
import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, average_precision_score, confusion_matrix, roc_curve

# This script is intended to replace D:\bigdadta\question2\analyze.py
R = Path(__file__).resolve().parents[1]
if not (R / '吉隆县历史灾害_干净版.csv').exists():
    # Also works when downloaded elsewhere and executed with project root as cwd.
    R = Path.cwd()
O = R / 'question2'
O.mkdir(parents=True, exist_ok=True)

TRAIN_END = '2021-07-31'
TEST_START = '2021-08-16'  # 15-day embargo, Aug 1 - 15
STUDY_START, STUDY_END = '2021-04-01', '2021-09-30'
BASE_COLS = ['P24', 'P360', 'D72']
COMPARE_COLS = {'P24_D72': ['P24', 'D72'], 'P72_D72': ['P72', 'D72'],
```

```
                'P360_D72': ['P360', 'D72']}


def safe_float(x):
    return float(x) if pd.notna(x) and np.isfinite(x) else None


def safe_metric(y, score, kind):
    if len(y) == 0 or len(np.unique(y)) < 2:
        return None
    return float(roc_auc_score(y, score) if kind == 'auc' else average_precision_score(y, score))


def fit_and_evaluate(train, test, cols, name):
    if train.empty or test.empty or train['y'].nunique() < 2:
        return {'name': name, 'features': cols, 'status': '数据不足，未能估计'}, None
    model = make_pipeline(StandardScaler(), LogisticRegression(
        C=0.1, class_weight='balanced', max_iter=2000, random_state=42))
    model.fit(train[cols], train['y'])
    tr_score = model.predict_proba(train[cols])[:, 1]
    fpr, tpr, thresholds = roc_curve(train['y'], tr_score)
    # sklearn includes an infinite threshold; prefer a finite threshold if tied.
    youden = tpr - fpr
    finite = np.isfinite(thresholds)
    candidate = np.flatnonzero(finite)
    if len(candidate) == 0:
        return {'name': name, 'features': cols, 'status': '训练阈值不可识别'}, None
    best = candidate[np.argmax(youden[candidate])]
    cut = float(thresholds[best])
    scores = model.predict_proba(test[cols])[:, 1]
    alert = scores >= cut
    tn, fp, fn, tp = confusion_matrix(test['y'], alert, labels=[0, 1]).ravel()
    scaler, clf = model.steps[0][1], model.steps[1][1]
    result = {
        'name': name, 'status': 'ok', 'features': cols,
        'train_n': int(len(train)), 'train_positive_days': int(train['y'].sum()),
        'test_n': int(len(test)), 'test_positive_days': int(test['y'].sum()),
        'threshold_from_training': cut,
        'test_auc': safe_metric(test['y'], scores, 'auc'),
        'test_ap': safe_metric(test['y'], scores, 'ap'),
        'test_prevalence': float(test['y'].mean()),
        'test_confusion_tn_fp_fn_tp': [int(tn), int(fp), int(fn), int(tp)],
        'test_event_recall': float(tp / (tp + fn)) if tp + fn else None,
        'test_background_alarm_rate': float(fp / (fp + tn)) if fp + tn else None,
        'scaler_mean': scaler.mean_.tolist(), 'scaler_scale': scaler.scale_.tolist(),
        'coefficients_standardized': clf.coef_[0].tolist(),
```

```
        'intercept_standardized': float(clf.intercept_[0]),
        'note': '模型输出为病例—背景条件评分，不是校准后的真实灾害概率。'
    }
    daily = pd.DataFrame({'date': test.index.strftime('%Y-%m-%d'),
                          'y': test['y'].to_numpy(), 'score': scores,
                          'alert': alert.astype(int)})
    daily.to_csv(O / f'temporal_holdout_{name}.csv', index=False, encoding='utf-8-sig')
    return result, model


def glacier_diagnostics():
    path = R / 'glacier.nc'
    result = {'file': str(path), 'status': 'not_found',
              'interpretation': '全文件统计不能替代已确认冰崩释放区统计，也不能用作 2021 年事件标签。'}
    if not path.exists():
        return result
    try:
        import xarray as xr
        with xr.open_dataset(path) as ds:
            result['status'] = 'ok'
            result['dataset_dimensions'] = {str(k): int(v) for k, v in ds.sizes.items()}
            result['variables'] = {}
            for name in ['thickness', 'v', 'slope']:
                if name not in ds:
                    result['variables'][name] = {'status': 'missing'}
                    continue
                da = ds[name]
                # Preserve file-defined missing value masks where xarray decodes them.
                arr = np.asarray(da.values, dtype=float)
                arr[~np.isfinite(arr)] = np.nan
                for attr in ['_FillValue', 'missing_value']:
                    val = da.attrs.get(attr)
                    if val is not None and np.isscalar(val):
                        arr[arr == float(val)] = np.nan
                # This is an explicit heuristic, not instrument quality control.
                arr[arr <= -999] = np.nan
                if name in ('thickness', 'v', 'slope'):
                    arr[arr < 0] = np.nan
                valid = arr[np.isfinite(arr)]
                result['variables'][name] = {
                    'status': 'ok' if valid.size else 'no_valid_cells',
                    'valid_cells': int(valid.size), 'units_attribute': da.attrs.get('units'),
                    'min': safe_float(np.min(valid)) if valid.size else None,
                    'p10': safe_float(np.percentile(valid, 10)) if valid.size else None,
                    'median': safe_float(np.median(valid)) if valid.size else None,
                    'p90': safe_float(np.percentile(valid, 90)) if valid.size else None,
```

```
                'max': safe_float(np.max(valid)) if valid.size else None,
            }
    except Exception as ex:
        result['status'] = 'error'
        result['error'] = f'{type(ex).__name__}: {ex}'
    return result


def main():
    event_path = R / '吉隆县历史灾害_干净版.csv'
    if not event_path.exists():
        raise FileNotFoundError(f'未找到灾害台账: {event_path}')
    events = pd.read_csv(event_path)
    events['date'] = pd.to_datetime(events['发生日期'], errors='coerce').dt.normalize()
    events = events.dropna(subset=['date'])
    met_files = sorted((R / 'data').glob('MET_*.csv'))
    if not met_files:
        raise FileNotFoundError(f'未找到气象文件: {R / "data" / "MET_*.csv"}')
    met = pd.concat([pd.read_csv(p) for p in met_files], ignore_index=True)
    met.index = pd.to_datetime(met.pop('Timestamp'), format='mixed', errors='coerce')
    met = met.loc[met.index.notna()].sort_index()
    if met.index.duplicated().any():
        raise ValueError('MET 时间戳存在重复，需先核对重复来源，不能静默去重。')
    p = pd.to_numeric(met['Precip(mm)'], errors='coerce')
    t = pd.to_numeric(met['Ta_2m(degree C)'], errors='coerce')
    p = p.where((p >= 0) & (p < 999))
    t = t.where((t > -80) & (t < 60))
    h = pd.DataFrame({'P': p, 'T': t}).asfreq('h')
    features = pd.DataFrame(index=h.index)
    for n in [24, 72, 360]:
        features[f'P{n}'] = h['P'].rolling(n, min_periods=n).sum().shift(1)
    features['D72'] = h['T'].clip(lower=0).rolling(72, min_periods=72).sum().shift(1) / 24
    # Only midnight predictions; no weather observations from the event day.
    daily = features.loc[features.index.hour == 0].copy()
    event_days = pd.DatetimeIndex(events['date'].unique())
    daily['y'] = daily.index.normalize().isin(event_days).astype(int)
    daily.to_csv(0 / 'daily_features.csv', encoding='utf-8-sig')
    audit = {
        'records': int(len(events)),
        'records_by_year': {str(k): int(v) for k, v in events['date'].dt.year.value_counts().sort_index().items()},
        'unique_event_days': int(events['date'].nunique()),
        'events_2021': int(events.loc[events['date'].dt.year == 2021, 'date'].nunique()),
        'timestamp_assumption': '保留原始 shift(1)：假设 Timestamp 为小时开始标签；若为小时结束标签需按元数据重新核对。
',
        'qc_note': '仅执行简单数值范围过滤；不插值。气象仪器 QC、时区和小时累计语义尚待核验。',
        'glacier_gate_in_dynamic_model': False,
```

```
    }
    v = daily.loc[STUDY_START:STUDY_END].copy()
    audit['missing_event_features'] = [str(x.date()) for x in v.index[(v.y == 1) & v[BASE_COLS].isna().any(axis=1)]]
    # Compare all models on the same complete-case dates, to keep metrics comparable.
    common_cols = sorted(set(BASE_COLS).union(*[set(c) for c in COMPARE_COLS.values()]))
    v = v.dropna(subset=common_cols)
    v.to_csv(0 / 'model_samples.csv', encoding='utf-8-sig')
    train = v.loc[:TRAIN_END].copy()
    test = v.loc[TEST_START:].copy()
    if train.empty or test.empty:
        raise ValueError('训练集或测试集为空，请检查时间覆盖和气象缺测。')
    audit['split'] = {'train_end': TRAIN_END, 'test_start': TEST_START,
                      'embargo': '2021-08-01 through 2021-08-15',
                      'common_complete_case_features': common_cols}
    audit['train_n'] = int(len(train))
    audit['train_positive_days'] = int(train.y.sum())
    audit['test_n'] = int(len(test))
    audit['test_positive_days'] = int(test.y.sum())
    baseline, baseline_model = fit_and_evaluate(train, test, BASE_COLS, 'baseline_P24_P360_D72')
    audit['baseline'] = baseline
    audit['comparisons'] = {}
    for name, cols in COMPARE_COLS.items():
        result, _ = fit_and_evaluate(train, test, cols, name)
        audit['comparisons'][name] = result
    # Keep the original filename for the baseline holdout as well.
    source = 0 / 'temporal_holdout_baseline_P24_P360_D72.csv'
    if source.exists():
        (0 / 'temporal_holdout.csv').write_bytes(source.read_bytes())
    if baseline_model is not None:
        case_date = pd.Timestamp('2026-08-26')
        if case_date in daily.index:
            case = daily.loc[case_date, BASE_COLS]
            audit['case_2026_features'] = {k: safe_float(v) for k, v in case.items()}
            if case.notna().all():
                score = baseline_model.predict_proba(pd.DataFrame([case], columns=BASE_COLS))[:, 1][0]
                audit['case_2026_score'] = float(score)
                audit['case_2026_alert'] = bool(score >= baseline['threshold_from_training'])
        audit['case_2026_note'] = '2026 年案例仅用于外部气象条件诊断；未参与训练，也不验证冰崩预测。'
    audit['glacier_diagnostics'] = glacier_diagnostics()
    positive = v.loc[v.y == 1, BASE_COLS]
    if not positive.empty:
        positive.describe(percentiles=[0.1, 0.5, 0.9]).to_csv(0 / 'event_feature_distribution.csv', encoding='utf-8-
sig')
    (0 / 'results_modified.json').write_text(json.dumps(audit, ensure_ascii=False, indent=2, allow_nan=False),
encoding='utf-8')
    rows = []
```

```
    for result in [baseline, *audit['comparisons'].values()]:
        rows.append({k: result.get(k) for k in ['name', 'status', 'train_n', 'train_positive_days',
                    'test_n', 'test_positive_days', 'threshold_from_training', 'test_auc',
                    'test_ap', 'test_event_recall', 'test_background_alarm_rate']})
    pd.DataFrame(rows).to_csv(O / 'model_comparison.csv', index=False, encoding='utf-8-sig')
    print(json.dumps(audit, ensure_ascii=False, indent=2, allow_nan=False))
    print('\n模型对比表：', O / 'model_comparison.csv')
    print('完整结果：', O / 'results_modified.json')

if __name__ == '__main__':
    main()
```

#Q1

```
    # -*- coding: utf-8 -*-
"""
问题一 V8.1 方案B：去日变化相关 + 方向修正 + 最终五因素（边界层用 0.3P+0.7M）

本版相对上一版 V8.1 的唯一改动：
- 边界层结构与湍流组选代表时，综合公式由 0.6*P+0.4*M 改为 0.3*P+0.7*M，
  使感热通量优先于温度梯度 2-20m，避免温度梯度分布窄导致 z 虚高。

输入：
- clead.csv
- Question1_Model_Results_V8/04_事件与历史同期对照_含敏感性对照.csv
- Question1_Model_Results_V8/05_暖季多窗口滚动分布摘要.csv
- Question1_Model_Results_V8/11_候选因素综合评分表.csv

输出：
- Question1_Model_Results_V8_1/
"""

from pathlib import Path
import warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

warnings.filterwarnings("ignore", category=RuntimeWarning)

BASE_DIR = Path(__file__).resolve().parent
INPUT_FILE = BASE_DIR / "clead.csv"
V8_DIR = BASE_DIR / "Question1_Model_Results_V8"
OUT_DIR = BASE_DIR / "Question1_Model_Results_V8_1"
```

```
EVENT_TIME = pd.Timestamp("2026-08-26 10:00:00")
HIST_YEARS = [2021, 2022, 2023, 2024, 2025]
WINDOWS = [6, 24, 48, 72]
WARM_MONTHS = [6, 7, 8, 9]

CANDIDATES = {
    "气温 2m": ["MET_Ta_2m(degree C)", "MET_Ta_2m", "Ta_2m"],
    "相对湿度": ["MET_RH_2m(%)", "MET_RH_2m", "RH_2m"],
    "降水": ["MET_Precip(mm)", "MET_Precip", "Precip(mm)"],
    "VPD": ["FLUX_VPD(hPa)", "FLUX_VPD", "VPD(hPa)"],
    "气压": ["FLUX_PA(kPa)", "FLUX_PA", "PA(kPa)"],
    "风速 2m": ["MET_WS_2m(m/s)", "MET_WS_2m", "WS_2m"],
    "风速 10m": ["MET_WS_10m(m/s)", "MET_WS_10m", "WS_10m"],
    "风速 20m": ["MET_WS_20m(m/s)", "MET_WS_20m", "WS_20m"],
    "温度 1m": ["MET_Ta_1m(degree C)", "MET_Ta_1m", "Ta_1m"],
    "温度 10m": ["MET_Ta_10m(degree C)", "MET_Ta_10m", "Ta_10m"],
    "温度 20m": ["MET_Ta_20m(degree C)", "MET_Ta_20m", "Ta_20m"],
    "感热通量": ["FLUX_H(W/m2)", "FLUX_H", "H(W/m2)"],
    "潜热通量": ["FLUX_LE(W/m2)", "FLUX_LE", "LE(W/m2)"],
    "湍流动能": ["FLUX_TKE(m2/s2)", "FLUX_TKE", "TKE"],
    "摩擦速度": ["FLUX_ustar(m/s)", "FLUX_u_star(m/s)", "FLUX_ustar"],
    "短波向下": ["RADM_Rsd(W/m2)", "RADM_Rsd", "Rsd(W/m2)"],
    "短波向上": ["RADM_Rsu(W/m2)", "RADM_Rsu", "Rsu(W/m2)"],
    "长波向下": ["RADM_Rld(W/m2)", "RADM_Rld", "Rld(W/m2)"],
    "长波向上": ["RADM_Rlu(W/m2)", "RADM_Rlu", "Rlu(W/m2)"],
    "土壤含水量": ["SOIL_VWC_0.8m(m3/m3)", "SOIL_VWC_0.8m", "VWC_0.8m"],
    "土壤热通量": ["SOIL_G_0.1m(W/m2)", "SOIL_G_0.1m", "G_0.1m"],
}

GROUPS = {
    "温度与融化背景": ["气温 2m", "气温 1 小时变化", "正积温 24h"],
    "水汽条件": ["VPD", "相对湿度"],
    "辐射与能量条件": ["净辐射", "短波向下", "长波向上", "长波向下", "短波向上"],
    "边界层结构与湍流": ["温度梯度 2-10m", "温度梯度 2-20m", "风速差 20-2m",
                         "湍流动能", "摩擦速度", "感热通量", "潜热通量"],
    "降水过程": ["降水"],
    "下垫面条件": ["土壤含水量", "土壤热通量"],
}

EXPECTED_DIRECTION = {
    "正积温 24h": +1, "气温 2m": +1, "气温 1 小时变化": +1,
    "VPD": +1, "相对湿度": -1,
    "净辐射": +1, "短波向下": +1, "长波向下": +1, "长波向上": +1, "短波向上": +1,
    "感热通量": +1, "潜热通量": +1, "湍流动能": +1, "摩擦速度": +1,
    "温度梯度 2-10m": +1, "温度梯度 2-20m": +1, "风速差 20-2m": +1,
    "降水": +1, "土壤含水量": +1, "土壤热通量": +1,
```

```
    "气压": 0,
}

PHYSICAL_SCORE = {
    "VPD": 0.95, "相对湿度": 0.90,
    "气温 2m": 0.95, "正积温 24h": 0.90, "气温 1 小时变化": 0.75,
    "净辐射": 0.85, "短波向下": 0.80, "长波向上": 0.75,
    "长波向下": 0.70, "短波向上": 0.70,
    "感热通量": 0.80, "潜热通量": 0.75,
    "湍流动能": 0.75, "摩擦速度": 0.70,
    "温度梯度 2-10m": 0.75, "温度梯度 2-20m": 0.70,
    "风速差 20-2m": 0.70,
    "降水": 0.60, "土壤含水量": 0.60, "土壤热通量": 0.55,
    "气压": 0.40,
}

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial Unicode MS", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False


def find_time_col(df):
    for c in ["Timestamp", "timestamp", "datetime", "DateTime", "time", "时间"]:
        if c in df.columns:
            return c
    hits = [c for c in df.columns if "timestamp" in str(c).lower() or "datetime" in str(c).lower()]
    if hits:
        return hits[0]
    raise ValueError("未找到时间列")


def find_col(df, candidates):
    for c in candidates:
        if c in df.columns:
            return c
    for cand in candidates:
        if len(cand) < 5:
            continue
        hits = [c for c in df.columns if cand.lower() in str(c).lower()]
        if len(hits) == 1:
            return hits[0]
    return None


def num(s):
    return pd.to_numeric(s, errors="coerce").replace([np.inf, -np.inf], np.nan)
```

```
def build_derived(df, cols):
    out = df.copy()
    rad = ["短波向下", "短波向上", "长波向下", "长波向上"]
    if all(cols.get(x) for x in rad):
        out["净辐射"] = (
            num(out[cols["短波向下"]]) - num(out[cols["短波向上"]])
            + num(out[cols["长波向下"]]) - num(out[cols["长波向上"]])
        )
    if cols.get("温度 10m") and cols.get("气温 2m"):
        out["温度梯度 2-10m"] = (num(out[cols["温度 10m"]]) - num(out[cols["气温 2m"]])) / 8
    if cols.get("温度 20m") and cols.get("气温 2m"):
        out["温度梯度 2-20m"] = (num(out[cols["温度 20m"]]) - num(out[cols["气温 2m"]])) / 18
    if cols.get("风速 20m") and cols.get("风速 2m"):
        out["风速差 20-2m"] = num(out[cols["风速 20m"]]) - num(out[cols["风速 2m"]])
    if cols.get("气温 2m"):
        t = num(out[cols["气温 2m"]])
        out["气温 1 小时变化"] = t.diff()
        out["正积温 24h"] = t.clip(lower=0).rolling(24, min_periods=18).sum()
    return out


def deseanonalize(warm_df, var_list):
    out = warm_df.copy()
    for v in var_list:
        if v not in out.columns:
            continue
        out[v] = num(out[v])
        if out[v].notna().sum() < 100:
            continue
        hourly_med = out.groupby(out["_time"].dt.hour)[v].transform("median")
        out[v + "_去日变化"] = out[v] - hourly_med
    return out


def main():
    if not V8_DIR.exists():
        raise FileNotFoundError(f"找不到 {V8_DIR}，请先运行 V8。")
    if not INPUT_FILE.exists():
        raise FileNotFoundError(f"找不到 {INPUT_FILE}。")

    OUT_DIR.mkdir(parents=True, exist_ok=True)

    comp = pd.read_csv(V8_DIR / "04_事件与历史同期对照_含敏感性对照.csv", encoding="utf-8-sig")
    season = pd.read_csv(V8_DIR / "05_暖季多窗口滚动分布摘要.csv", encoding="utf-8-sig")
    score11 = pd.read_csv(V8_DIR / "11_候选因素综合评分表.csv", encoding="utf-8-sig")
```

```
comp_s = comp[comp["版本"] == "范围筛查敏感性版本"].copy()
season_s = season[season["版本"] == "范围筛查敏感性版本"].copy()

try:
    df = pd.read_csv(INPUT_FILE, encoding="utf-8-sig", low_memory=False)
except UnicodeDecodeError:
    df = pd.read_csv(INPUT_FILE, encoding="gbk", low_memory=False)
time_col = find_time_col(df)
df["_time"] = pd.to_datetime(df[time_col], errors="coerce")
df = df.dropna(subset=["_time"]).sort_values("_time").drop_duplicates("_time").reset_index(drop=True)

cols = {name: find_col(df, cand) for name, cand in CANDIDATES.items()}
work = df.copy()
for name, col in cols.items():
    if col:
        work[name] = num(work[col])
work = build_derived(work, cols)

warm = work[
    work["_time"].dt.year.isin(HIST_YEARS)
    & work["_time"].dt.month.isin(WARM_MONTHS)
].copy()

vars_all = [v for v in score11["变量"].tolist() if v in warm.columns]
vars_all = [v for v in vars_all if warm[v].notna().sum() > 100]

warm_ds = deseanonalize(warm, vars_all)
ds_cols = [v + "_去日变化" for v in vars_all if v + "_去日变化" in warm_ds.columns]
corr_ds = warm_ds[ds_cols].corr(method="spearman")
corr_ds.index = [c.replace("_去日变化", "") for c in corr_ds.index]
corr_ds.columns = [c.replace("_去日变化", "") for c in corr_ds.columns]
corr_ds.to_csv(OUT_DIR / "01_去日变化 Spearman 相关矩阵.csv", encoding="utf-8-sig")

if len(corr_ds) >= 2:
    fig, ax = plt.subplots(figsize=(max(9, len(corr_ds) * 0.55), max(7, len(corr_ds) * 0.45)))
    im = ax.imshow(corr_ds.values, cmap="RdBu_r", vmin=-1, vmax=1)
    ax.set_xticks(range(len(corr_ds)))
    ax.set_yticks(range(len(corr_ds)))
    ax.set_xticklabels(corr_ds.columns, rotation=75, ha="right", fontsize=8)
    ax.set_yticklabels(corr_ds.index, fontsize=8)
    ax.set_title("去日变化后 Spearman 相关矩阵（2021—2025 暖季）")
    fig.colorbar(im, ax=ax, fraction=0.025, pad=0.02)
    fig.tight_layout()
    fig.savefig(OUT_DIR / "图_去日变化 Spearman 相关热图.png", dpi=180)
    plt.close(fig)
```

```
base = score11.copy()
base["物理因素组"] = base["变量"].apply(
    lambda v: next((g for g, members in GROUPS.items() if v in members), "其他")
)

ev72 = {}
hist72 = {}
for _, row in comp_s[comp_s["窗口小时"] == 72].iterrows():
    ev72[row["变量"]] = row["事件统计值"]
    hist72[row["变量"]] = row["历史同期中位数"]

pct72 = {}
z72 = {}
for _, row in season_s[season_s["窗口小时"] == 72].iterrows():
    pct72[row["变量"]] = row["暖季经验百分位_pct"]
    z72[row["变量"]] = row["暖季稳健异常分数"]

def direction_factor(var):
    exp = EXPECTED_DIRECTION.get(var, 0)
    ev = ev72.get(var, np.nan)
    hm = hist72.get(var, np.nan)
    if pd.isna(ev) or pd.isna(hm):
        return 0.5
    if var == "降水" and ev <= 0:
        return 0.0
    if exp == 0:
        return 0.5
    if ev == hm:
        return 0.5
    dev = 1 if ev > hm else -1
    return 1.0 if dev == exp else 0.2

base["预期方向"] = base["变量"].map(lambda v: EXPECTED_DIRECTION.get(v, 0))
base["方向一致系数"] = base["变量"].map(direction_factor)
base["P_方向修正"] = base["P_暖季异常强度"] * base["方向一致系数"]

# ===== 物理组内选代表 =====
rep_rows = []
for group, gdf in base.groupby("物理因素组"):
    gdf = gdf.copy()
    if group == "边界层结构与湍流":
        # ================= 方案 B 唯一修改处 =================
        # 原版：gdf["组内综合"] = 0.6 * gdf["P_方向修正"] + 0.4 * gdf["M_物理解释"]
        # 方案 B：把 M 权重提高到 0.7，使感热通量优先于温度梯度 2-20m
        gdf["组内综合"] = 0.3 * gdf["P_方向修正"] + 0.7 * gdf["M_物理解释"]
```

```
        # ==================================================
        rep = gdf.sort_values(["组内综合", "P_方向修正"], ascending=False).iloc[0]
    else:
        rep = gdf.sort_values(["P_方向修正", "S_综合评分"], ascending=False).iloc[0]
    rep_rows.append({
        "物理因素组": group,
        "代表变量": rep["变量"],
        "代表 P_方向修正": rep["P_方向修正"],
        "代表 S_原": rep["S_综合评分"],
        "代表 Q": rep["Q_数据质量"],
        "代表 C": rep["C_多窗口一致性"],
        "代表 M": rep["M_物理解释"],
        "组内成员": "; ".join(gdf["变量"].tolist()),
    })
reps = pd.DataFrame(rep_rows)
reps.to_csv(OUT_DIR / "02_物理组内代表变量.csv", index=False, encoding="utf-8-sig")

rep_vars = reps["代表变量"].tolist()
indep = {}
for v in rep_vars:
    if v not in corr_ds.columns:
        indep[v] = 0.5
        continue
    others = [r for r in rep_vars if r != v and r in corr_ds.columns]
    if not others:
        indep[v] = 1.0
        continue
    max_abs = float(corr_ds.loc[v, others].abs().max())
    indep[v] = max(0.0, 1.0 - max_abs)

base["I_修正"] = base["变量"].map(lambda v: indep.get(v, 0.0) if v in rep_vars else 0.0)
base["S_修正"] = (
    0.25 * base["Q_数据质量"]
    + 0.25 * base["P_方向修正"]
    + 0.20 * base["C_多窗口一致性"]
    + 0.15 * base["I_修正"]
    + 0.15 * base["M_物理解释"]
)

base = base.sort_values("S_修正", ascending=False)
base.to_csv(OUT_DIR / "03_方向与独立性修正后综合评分.csv", index=False, encoding="utf-8-sig")

# ===== 最终选五因素：排除“其他”组 =====
rep_base = base[base["变量"].isin(rep_vars)].copy()
rep_base = rep_base[rep_base["物理因素组"] != "其他"]
rep_base = rep_base.sort_values("S_修正", ascending=False)
```

```
def group_of(v):
    return next((g for g, members in GROUPS.items() if v in members), "其他")

rep_base["物理组"] = rep_base["变量"].apply(group_of)

chosen = []
used_groups = set()

def add_row(row):
    if row["物理组"] in used_groups:
        return False
    chosen.append(row.to_dict())
    used_groups.add(row["物理组"])
    return True

priority_groups = ["温度与融化背景", "水汽条件", "辐射与能量条件", "边界层结构与湍流"]
for g in priority_groups:
    sub = rep_base[rep_base["物理组"] == g]
    if len(sub):
        add_row(sub.iloc[0])
    if len(chosen) >= 5:
        break

for _, row in rep_base.iterrows():
    if len(chosen) >= 5:
        break
    if row["物理组"] in used_groups:
        continue
    if row["变量"] == "降水" and row["方向一致系数"] < 0.5:
        continue
    add_row(row)

if len(chosen) < 5:
    for _, row in rep_base.iterrows():
        if len(chosen) >= 5:
            break
        if any(c["变量"] == row["变量"] for c in chosen):
            continue
        chosen.append(row.to_dict())

final5 = pd.DataFrame(chosen[:5])
if not final5.empty:
    final5.insert(0, "候选顺序", range(1, len(final5) + 1))
    final5["方向解释"] = final5.apply(
        lambda r: f"事件72h={ev72.get(r['变量'], np.nan):.4g}, "
```

```
                f"历史中位数={hist72.get(r['变量'], np.nan):.4g}, "
                f"暖季百分位={pct72.get(r['变量'], np.nan):.1f}%"
            if pd.notna(ev72.get(r['变量'], np.nan)) else "数据不足",
            axis=1
        )
        final5["解释限制"] = (
            "方向与独立性修正后的候选名单；不是概率或因果排名；"
            "降水仅在事件降水>0 且方向一致时允许入选；已排除“其他”物理组（如气压）；"
            "边界层组按 0.3P+0.7M 综合选代表"
        )
    final5.to_csv(OUT_DIR / "04_方向修正后最终五因素.csv", index=False, encoding="utf-8-sig")

    # 三类覆盖
    if not final5.empty:
        three_rows = []
        for _, r in final5.iterrows():
            g = r["物理组"]
            if g == "边界层结构与湍流":
                cls = "大气边界层结构"
            elif g in ["温度与融化背景", "水汽条件", "降水过程"]:
                cls = "天气过程"
            elif g == "辐射与能量条件":
                cls = "极端气象事件"
            else:
                cls = "辅助/下垫面"
            three_rows.append({
                "三类归属": cls, "候选顺序": r["候选顺序"], "变量": r["变量"],
                "方向一致系数": r["方向一致系数"],
                "S_修正": r["S_修正"],
                "方向解释": r["方向解释"]
            })
        pd.DataFrame(three_rows).to_csv(
            OUT_DIR / "05_最终五因素三类覆盖.csv", index=False, encoding="utf-8-sig"
        )

    # 降水专项
    precip_rows = []
    for _, r in comp_s[comp_s["变量"] == "降水"].iterrows():
        s = season_s[
            (season_s["变量"] == "降水") & (season_s["窗口小时"] == r["窗口小时"])
        ]
        precip_rows.append({
            "窗口小时": r["窗口小时"],
            "事件降水统计值": r["事件统计值"],
            "历史同期中位数": r["历史同期中位数"],
            "暖季中位数": s["暖季中位数"].iloc[0] if len(s) else np.nan,
```

```
            "暖季经验百分位_pct": s["暖季经验百分位_pct"].iloc[0] if len(s) else np.nan,
            "方向一致系数": direction_factor("降水"),
            "结论": "事件窗口无降水，短时降水不是直接激发条件" if r["事件统计值"] <= 0 else "存在降水信号"
        })
    pd.DataFrame(precip_rows).to_csv(
        OUT_DIR / "06_降水专项结论.csv", index=False, encoding="utf-8-sig"
    )

    # 图：修正前后对比
    merged = base[["变量", "S_综合评分", "S_修正", "P_暖季异常强度", "P_方向修正", "方向一致系数"]].copy()
    merged = merged.sort_values("S_修正", ascending=False).head(20)
    fig, ax = plt.subplots(figsize=(11, 5))
    x = np.arange(len(merged))
    ax.bar(x - 0.2, merged["S_综合评分"], 0.4, label="V8 原 S")
    ax.bar(x + 0.2, merged["S_修正"], 0.4, label="V8.1 修正 S")
    ax.set_xticks(x)
    ax.set_xticklabels(merged["变量"], rotation=75, ha="right", fontsize=8)
    ax.set_ylabel("综合评分")
    ax.set_title("V8 原评分 vs V8.1 方案 B 修正评分（前 20）")
    ax.legend()
    ax.grid(axis="y", alpha=.25)
    fig.tight_layout()
    fig.savefig(OUT_DIR / "图_V8 与 V8_1 评分对比.png", dpi=180)
    plt.close(fig)

    if not final5.empty:
        fig, ax = plt.subplots(figsize=(9, 4.5))
        ax.barh(final5["变量"], final5["S_修正"], color="steelblue")
        for i, (_, r) in enumerate(final5.iterrows()):
            ax.text(r["S_修正"] + 0.005, i, f"方向系数={r['方向一致系数']:.1f}",
                    va="center", fontsize=8)
        ax.set_xlabel("修正后综合评分 S")
        ax.set_title("V8.1 方案 B 最终五因素（排除“其他” + 边界层 0.3P+0.7M）")
        ax.grid(axis="x", alpha=.25)
        fig.tight_layout()
        fig.savefig(OUT_DIR / "图_最终五因素_方向修正.png", dpi=180)
        plt.close(fig)

    readme = """问题一 V8.1 方案 B 输出说明
```

唯一修改：

- 边界层结构与湍流组综合公式由 0.6*P+0.4*M 改为 0.3*P+0.7*M，
  使感热通量优先于温度梯度 2-20m，避免温度梯度暖季分布窄导致 z 虚高。

文件说明：

01_去日变化 Spearman 相关矩阵.csv

02_物理组内代表变量.csv（边界层组应为感热通量）

03_方向与独立性修正后综合评分.csv

04_方向修正后最终五因素.csv（预期：正积温、VPD、净辐射、感热通量、土壤热通量）

05_最终五因素三类覆盖.csv

06_降水专项结论.csv

限制：

1. 方向系数为物理先验，不是统计估计。

2. 最终五因素为关联候选，不证明唯一因果。

"""

```
    (OUT_DIR / "README_V8_1_方案B说明.txt").write_text(readme, encoding="utf-8")

    print("V8.1 方案B运行完成。")
    print(f"输出：{OUT_DIR}")
    print("\n优先查看：")
    print("02_物理组内代表变量.csv")
    print("04_方向修正后最终五因素.csv")
    print("05_最终五因素三类覆盖.csv")
    print("06_降水专项结论.csv")



if __name__ == "__main__":
    main()
```

#Q3

```
    # -*- coding: utf-8 -*-
"""第三问收尾：固定日00:00发布 + 前向测试 + 条件95%区间审计。
输入：Q3_stage1_results/{hourly_q2_score.csv,event_date_audit.csv,test_daily_scores.csv}
    Q3_stage2_results/train_daily_calibration.csv
仅依赖 pandas/numpy；无 matplotlib/rasterio。直接放在Q3目录运行。
"""
from pathlib import Path
import argparse, json
import numpy as np
import pandas as pd

B0=-2.070474371019093; B1=0.4419896098872232
H=48; TARGET_BURDEN=0.35

def forecast(score, issue):
    v=np.clip(float(score),1e-10,1-1e-10)
    p24=float(1/(1+np.exp(-np.clip(B0+B1*np.log(v/(1-v)),-30,30))))
    h=1-(1-p24)**(1/24)
    pm=h*(1-h)**np.arange(H)
    prob=float(pm.sum()); cdf=np.cumsum(pm)/prob
    lo=int(np.searchsorted(cdf,.025)); hi=int(np.searchsorted(cdf,.975))
```

```
    return p24,prob,issue+pd.Timedelta(hours=lo),issue+pd.Timedelta(hours=hi+1)

def interval_audit(a,b,d):
    e=d+pd.Timedelta(days=1)
    if a<=d and b>=e:return 'certain_covered'
    if b<=d or a>=e:return 'certain_not_covered'
    return 'unknown_partial_overlap'

def main():
    global B0, B1
    ap=argparse.ArgumentParser(); root=Path(__file__).resolve().parent
    ap.add_argument('--root',default=str(root)); args=ap.parse_args(); root=Path(args.root)
    s1=root/'Q3_stage1_results'; s2=root/'Q3_stage2_results';
out=root/'Q3_final_validation_results';out.mkdir(exist_ok=True)

fn=[s1/'hourly_q2_score.csv',s1/'event_date_audit.csv',s1/'test_daily_scores.csv',s2/'train_daily_calibration.csv']
    for f in fn:
        if not f.exists():raise FileNotFoundError(f'缺少必要文件: {f}')
    calibration=json.loads((s2/'stage2_summary.json').read_text())['calibration']
    B0=calibration['intercept']; B1=calibration['coef']
    hourly=pd.read_csv(fn[0],parse_dates=['Timestamp']).set_index('Timestamp').sort_index()
    ev=pd.read_csv(fn[1]);events={pd.Timestamp(x).normalize() for x in ev['event_date'].dropna()}
    train=pd.read_csv(fn[3]);test=pd.read_csv(fn[2]);
    # Keep both training and test release grids strictly within their original splits.
    def build(df,split):
        rows=[]
        for _,r in df.iterrows():
            d=pd.Timestamp(r['date']).normalize()
            if d not in hourly.index or pd.isna(hourly.at[d,'score']):continue
            p24,p48,a,b=forecast(hourly.at[d,'score'],d)
            matching=sorted([e for e in events if d<=e<d+pd.Timedelta(hours=H)])
            rows.append(dict(split=split,issue_time=d,score=float(hourly.at[d,'score']),p24=p24,p48=p48,
                label_event_day_within_48h=int(bool(matching)),events_in_window=';'.join(str(x.date()) for x in
matching),
                ci95_conditional_start=a,ci95_conditional_end_exclusive=b))
        return pd.DataFrame(rows)
    tr=build(train,'train');te=build(test,'test')
    # Explicitly select threshold with TRAIN ONLY. Candidate thresholds from train projected 48-hour scenario risks.
    thresholds=np.unique(np.r_[tr.p48.to_numpy(),tr.p48.max()+1e-9])
    choices=[]
    for th in thresholds:
        alarm=tr.p48>=th; bg=tr.label_event_day_within_48h==0; pos=~bg
        burden=float(alarm[bg].mean()) if bg.sum() else np.nan
        recall=float(alarm[pos].mean()) if pos.sum() else np.nan

choices.append(dict(threshold=float(th),train_event_window_recall=recall,train_background_window_alarm_fraction=burden
```

```
,
                                train_alarm_fraction=float(alarm.mean())))
    ch=pd.DataFrame(choices)
    feasible=ch[ch.train_background_window_alarm_fraction<=TARGET_BURDEN+1e-12]
    candidates=feasible if len(feasible) else ch

selected=candidates.sort_values(['train_event_window_recall','train_background_window_alarm_fraction','threshold'],asc
ending=[False,True,False]).iloc[0]
    th=float(selected.threshold)
    for df in (tr,te):df['warning']=df.p48>=th
    # Metrics window-level and event-day-level (nonindependence explicit).
    def metrics(df):
        pos=df.label_event_day_within_48h.astype(bool);neg=~pos;w=df.warning
        return dict(n_issued=len(df),n_positive_windows=int(pos.sum()),n_background_windows=int(neg.sum()),
            recall_positive_windows=float(w[pos].mean()) if pos.sum() else None,
            alarm_fraction_background_windows=float(w[neg].mean()) if neg.sum() else None,
            overall_warning_fraction=float(w.mean()) if len(df) else None,
            tp=int((w&pos).sum()),fp=int((w&neg).sum()),fn=int((~w&pos).sum()),tn=int((~w&neg).sum()))
    # Distinct event dates in TEST only, find earliest issued warning in prior 48h, excluding issues on event date
    test_dates=sorted(pd.to_datetime(test['date']).dt.normalize().unique())
    evtest=sorted(e for e in events if e in set(test_dates))
    erows=[]
    for e in evtest:
        preceding=te[(te.issue_time<e)&(te.issue_time+pd.Timedelta(hours=H)>e)&te.warning]
        # Note boundary condition: event might occur at any point during its recorded day.
        earliest=preceding.issue_time.min() if len(preceding) else pd.NaT
        # The earlier issue can be 24-48h ahead of date start; unknown true event hour adds [0,24)h.
        if pd.isna(earliest):

erows.append(dict(event_date=e,any_prior_warning=False,earliest_prior_warning='',lead_lower_hours='',lead_upper_exclus
ive_hours='',coverage_status='no_warning'))
        else:
            p=preceding[preceding.issue_time==earliest].iloc[0]
            min_lead=(e-earliest).total_seconds()/3600
            erows.append(dict(event_date=e,any_prior_warning=True,earliest_prior_warning=earliest,
                              lead_lower_hours=min_lead,lead_upper_exclusive_hours=min_lead+24,

coverage_status=interval_audit(pd.Timestamp(p.ci95_conditional_start),pd.Timestamp(p.ci95_conditional_end_exclusive),e
)))
    ea=pd.DataFrame(erows)
    # Audit every event mention by issue to keep coverage bounds conservative.
    coverage=[]
    for _,r in te.iterrows():
        if not r.events_in_window:continue
        for es in r.events_in_window.split(';'):
            e=pd.Timestamp(es)
```

```
            coverage.append(dict(issue_time=r.issue_time,event_date=e,warning=bool(r.warning),

status=interval_audit(pd.Timestamp(r.ci95_conditional_start),pd.Timestamp(r.ci95_conditional_end_exclusive),e)))
    cov=pd.DataFrame(coverage)
    counts=cov.status.value_counts().to_dict() if len(cov) else {}
    total=len(cov); lower=counts.get('certain_covered',0)/total if total else None
    upper=(counts.get('certain_covered',0)+counts.get('unknown_partial_overlap',0))/total if total else None
    # Daily warning streak lengths in test (not hourly 1442h run).
    te=te.sort_values('issue_time').reset_index(drop=True)
    groups=(te.issue_time.diff().dt.days.ne(1)|te.warning.ne(te.warning.shift())).cumsum()
    runs=[]
    for _,g in te.groupby(groups):
        if
bool(g.warning.iloc[0]):runs.append(dict(start=str(g.issue_time.iloc[0]),end_inclusive=str(g.issue_time.iloc[-
1]),n_daily_issues=len(g)))

pd.DataFrame(runs,columns=['start','end_inclusive','n_daily_issues']).to_csv(out/'warning_runs_test.csv',index=False,e
ncoding='utf-8-sig')
    pd.concat([tr,te]).to_csv(out/'issued_forecasts.csv',index=False,encoding='utf-8-sig')
    ch.to_csv(out/'train_threshold_candidates.csv',index=False,encoding='utf-8-sig')
    ea.to_csv(out/'test_distinct_event_audit.csv',index=False,encoding='utf-8-sig')
    cov.to_csv(out/'date_interval_coverage_audit.csv',index=False,encoding='utf-8-sig')
    result=dict(policy=dict(issue='daily
00:00',horizon_hours=48,threshold=th,train_background_alarm_fraction_target=TARGET_BURDEN,
            selected_on='2021 train only',warning_semantics='issue-specific decision, not new crossing; warning if
p48 >= threshold'),
            train=metrics(tr),test=metrics(te),n_test_distinct_event_days=len(ea),
            n_event_days_with_prior_warning=int(ea.any_prior_warning.sum()) if len(ea) else 0,
            n_test_daily_warning_runs=len(runs),maximum_test_consecutive_warning_days=max([x['n_daily_issues'] for x
in runs],default=0),

conditional_95_interval_date_coverage_lower_bound=lower,conditional_95_interval_date_coverage_upper_bound=upper,
            n_overlapping_event_window_mentions=total,coverage_status_counts=counts,
            warnings=['日期级事件起始小时不明确；上下界不是95%覆盖率的估计。','48h重叠窗口不独立；不要把窗口数当作独
立灾害次数。',
              '事件日可能漏报；背景窗口报警比例不是正式误报率。',
              '天气使用风险持续情景，并非真实历史天气预报。',
              '风险校准基于少量训练日，预测区间是条件分布，不代表真实95%可靠度。',
              '2026年冰崩链式事件不作为泥石流精确触发时间的校准标签。'])
    (out/'final_validation_summary.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
    print('[完成] 输出:',out)
    print('训练期选定p48阈值:',round(th,6),'训练窗口指标:',metrics(tr))
    print('测试窗口指标:',metrics(te))
    print('独立事件日期:',len(ea),'事前至少一次报警:',result['n_event_days_with_prior_warning'])
    print('时间区间日期可判定覆盖上下界:',lower,upper,'注意重叠窗口不独立。')
```

```
    print('时间小时级 95%覆盖率未获验证。')
if __name__=='__main__':main()
```

#Q4

```
# -*- coding: utf-8 -*-
"""吉隆泥石流三维动力学仿真 | 启动：python -m streamlit run app.py"""
from __future__ import annotations
import json
from pathlib import Path
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import streamlit as st
from q4_solver import Config, load_terrain, simulate, PORT
from academic_visualization import make_animation_html
from q4_viz import save_all

ROOT=Path(__file__).resolve().parent
DEM=ROOT/'data'/'elevation_utm45n.tif'
RESULTS=ROOT/'results'

st.set_page_config(page_title='吉隆泥石流三维动力学仿真',layout='wide',initial_sidebar_state='expanded')
st.markdown('''<style>
html,body,[class*="css"]{font-family:"Microsoft YaHei","PingFang SC",sans-serif}
.block-container{padding-top:1.35rem;padding-bottom:2.8rem;max-width:1660px}
h1{font-weight:650;letter-spacing:.3px;font-size:2rem}h2,h3{letter-spacing:.1px}
[data-testid="stMetric"]{background:#f7f8f9;border:1px solid #e0e5e8;border-radius:7px;padding:13px 17px}
[data-testid="stMetricLabel"]{color:#556574}
[data-testid="stSidebar"]{border-right:1px solid #dfe4e8}
</style>''',unsafe_allow_html=True)

st.title('吉隆口岸冰岩崩—泥石流三维动力学仿真')
st.caption('二维深度平均动力学 / Voellmy 摩擦阻力 / 侵蚀与沉积 / 三维地形可视化')

@st.cache_resource(show_spinner=False)
def terrain_data(grid):
    return load_terrain(DEM,Config(grid_m=float(grid)))

@st.cache_data(show_spinner=False)
def load_precomputed():
    folder=RESULTS
    report=json.loads((folder/'模型参数和结果.json').read_text('utf-8'))
    with np.load(folder/'基准情景逐帧.npz',allow_pickle=False) as z:
        frames=[a.copy() for a in z['frames']]
        times=z['times'].tolist()
        hmax=z['hmax'].copy();vmax=z['vmax'].copy()
```

```
    df=pd.read_csv(folder/'模拟过程时序.csv')
    return dict(cfg=Config(**report['模型参数假设']),frames=frames,times=times,
                hmax=hmax,vmax=vmax,records=df.to_dict('records'),precomputed=True,
                first_port_frame_s=report.get('首个满足口岸300米邻域的保存帧时间_s'))

st.sidebar.subheader('数值模拟控制台')
st.sidebar.caption('物源与动力学参数')
initial_volume=st.sidebar.number_input('初始物源体积  m³',min_value=1.e7,max_value=5.e8,
                                       value=1.75e8,step=5.e6,format='%.0f')
c1,c2=st.sidebar.columns(2)
with c1:
    mu=st.number_input('初期摩擦  μ₀ ',min_value=0.,max_value=.50,value=.060,step=.005,format='%.3f')
with c2:
    mu_late=st.number_input('后期摩擦  μ₁',min_value=0.,max_value=.50,value=.010,step=.005,format='%.3f')
xi=st.sidebar.number_input('Voellmy阻力参数  ξ',min_value=100.,max_value=12000.,value=5000.,step=100.,format='%.0f')
k=st.sidebar.number_input('经验侵蚀系数  k',min_value=0.,max_value=.02,value=.000678,step=.00002,format='%.6f')
fluid=st.sidebar.number_input('阻力转换时间  s',min_value=0,max_value=300,value=100,step=10)
with st.sidebar.expander('源区与侵蚀设置'):
    length=st.number_input('释放区长度  m',min_value=400.,max_value=4000.,value=2040.,step=100.)
    width=st.number_input('释放区宽度  m',min_value=400.,max_value=4000.,value=2040.,step=100.)
    vcrit=st.number_input('侵蚀临界速度  m/s',min_value=0.,max_value=60.,value=15.,step=1.)
    erocap=st.number_input('可侵蚀厚度  m',min_value=0.,max_value=100.,value=25.,step=5.)
    depos=st.number_input('沉积系数  s⁻¹',min_value=0.,max_value=.005,value=.0003,step=.0001,format='%.4f')
st.sidebar.divider()
st.sidebar.caption('时间与网格')
grid=st.sidebar.selectbox('计算网格  m',[90,60,45,120],index=0)
duration=st.sidebar.slider('计算时长  s',120,1200,600,60)
snapshot=st.sidebar.selectbox('过程存储间隔  s',[10,20,30,60],index=0)
stride=st.sidebar.select_slider('三维地形显示密度',options=['标准','精细'],value='标准')
run=st.sidebar.button('开始计算',type='primary',width='stretch')
st.sidebar.caption('调整参数后点击“开始计算”更新图表。')

cfg=Config(initial_volume_m3=float(initial_volume),source_length_m=float(length),source_width_m=float(width),
           grid_m=float(grid),duration_s=float(duration),snapshot_s=float(snapshot),
           mu=float(mu),mu_late=float(mu_late),xi_mps2=float(xi),erosion_k=float(k),
           fluidization_after_s=float(fluid),erosion_speed_threshold_mps=float(vcrit),
           erodible_depth_m=float(erocap),deposit_k_per_s=float(depos))
if run:
    with st.status('正在执行二维有限体积计算',expanded=True) as status:
        prog=st.progress(0.0,text='网格与初始条件')
        try:
            T=terrain_data(grid)
            result=simulate(T,cfg,progress=lambda f:prog.progress(min(f,1.),text=f'求解进度
{f:.0%}'),max_steps=15000)
        except Exception as e:
            status.update(label='计算未完成',state='error')
```

```
            st.error(f'{type(e).__name__}：{e}')
        else:
            st.session_state['current_result']=result
            status.update(label='计算完成',state='complete',expanded=False)
            prog.empty()

R=st.session_state.get('current_result')
if R is None:
    try:
        R=load_precomputed()
    except Exception as exc:
        st.error('尚未发现预计算结果，请运行 python run_batch.py。')
        st.code(str(exc))
        st.stop()
T=terrain_data(R['cfg'].grid_m)
last=R['records'][-1]
arrival=R.get('first_port_frame_s')
if arrival is None:
    hits=[a['时间_s'] for a in R['records'] if a['口岸300米邻域是否达到']]
    arrival=min(hits) if hits else None
cols=st.columns(5)
cols[0].metric('累计扩展距离',f'{last["最大累计直线距离_km"]:.2f} km')
cols[1].metric('最大速度',f'{last["全程峰值速度_mps"]:.1f} m/s')
cols[2].metric('口岸邻域进入时刻',f'{arrival:.0f} s' if arrival is not None else '—')
cols[3].metric('累计侵蚀体积',f'{last["累计侵蚀体积_m3"]/1e6:.1f} 百万 m³')
cols[4].metric('累计影响面积',f'{last["累计影响面积_km2"]:.2f} km²')

view,process,plan,export=st.tabs(['三维仿真','动力学过程','流深空间分布','数据与导出'])

with view:
    html=make_animation_html(T,R,stride=2 if stride=='标准' else 1)
    if hasattr(st,'iframe'):
        st.iframe(html,height=815,width='stretch',alt='泥石流三维地形动态仿真')
    else:
        import streamlit.components.v1 as components
        components.html(html,height=815,scrolling=False)
    st.download_button('下载三维动画网页',data=html.encode('utf-8'),
                       file_name='吉隆泥石流_三维动画.html',mime='text/html')

with process:
    df=pd.DataFrame(R['records'])
    left,right=st.columns(2)
    def line_chart(series,title,ytitle):
        fig=go.Figure()
        for label,y in series:
            fig.add_trace(go.Scatter(x=df['时间_s'],y=y,mode='lines',name=label,line=dict(width=2.2)))
```

```
        fig.update_layout(title=dict(text=title,font=dict(size=15)),
            template='plotly_white',height=320,margin=dict(l=30,r=15,b=40,t=47),
            xaxis_title='模拟时间 / s',yaxis_title=ytitle,legend=dict(orientation='h',y=-.28))
        return fig
    with left:
        st.plotly_chart(line_chart([('流速',df['当前速度最大值_mps'])],
                    '流动速度时序','速度 / m·s⁻¹'),width='stretch')
        st.plotly_chart(line_chart([('侵蚀',df['累计侵蚀体积_m3']/1e6),
                                    ('沉积',df['累计沉积体积_m3']/1e6)],
                    '累计侵蚀与沉积','体积 / 百万 m³'),width='stretch')
    with right:
        st.plotly_chart(line_chart([('最大流深',df['当前流深最大值_m'])],
                    '流深变化','流深 / m'),width='stretch')
        st.plotly_chart(line_chart([('距离口岸',df['距口岸最近有效流动单元_m']/1000)],
                    '前锋空间位置','距离 / km'),width='stretch')
    with st.expander('查看质量收支时序'):
        st.plotly_chart(line_chart([('质量收支误差',df['物质量平衡相对误差']*100)],
                    '物质收支误差','误差 / %'),width='stretch')
    st.dataframe(df,width='stretch',hide_index=True)
    st.download_button('下载全过程 CSV',df.to_csv(index=False).encode('utf-8-sig'),
                       file_name='模拟过程时序.csv',mime='text/csv')

with plan:
    selected=st.slider('模拟时刻',min_value=0,max_value=len(R['times'])-1,value=len(R['times'])//2)
    H=R['frames'][selected]
    wet=H>=R['cfg'].wet_threshold_m
    z=np.where(wet,H,np.nan)
    Z=np.where(T.valid,T.z0,np.nan)
    from matplotlib.colors import LightSource
    from scipy.ndimage import distance_transform_edt
    zfill=Z.copy();valid=np.isfinite(zfill)
    if not valid.all():
        ids=distance_transform_edt(~valid,return_distances=False,return_indices=True)
        zfill[~valid]=zfill[tuple(ids)][~valid]
    hill=LightSource(azdeg=315,altdeg=50).hillshade(zfill,vert_exag=.6,dx=T.dx,dy=T.dy)
    fig=go.Figure(go.Heatmap(x=T.x,y=T.y,z=hill,colorscale='Greys',showscale=False,hoverinfo='skip'))
    fig.add_trace(go.Heatmap(x=T.x,y=T.y,z=z,colorscale='YlOrRd',opacity=.89,
                             colorbar=dict(title='流深 / m'),zmin=0,hovertemplate='%{z:.1f} m<extra>流深</extra>'))
    marks=[('冰崩点',(R['cfg'].source_x,R['cfg'].source_y),'#e5bb75'),
           ('吉隆口岸',PORT,'#4db7d5')]
    for name,coord,color in marks:
        fig.add_trace(go.Scatter(x=[coord[0]],y=[coord[1]],mode='markers+text',
                      text=[name],textposition='top center',name=name,
                      marker=dict(size=10,color=color,line=dict(width=1,color='black'))))
    fig.update_layout(template='plotly_white',height=650,
                      title=f'流深空间分布   t = {R["times"][selected]:.0f} s',
```

```
                    xaxis=dict(title='UTM 东坐标 / m'),
                    yaxis=dict(title='UTM 北坐标 / m',scaleanchor='x'),margin=dict(l=50,r=30,t=65,b=50))
    st.plotly_chart(fig,width='stretch')

with export:
    conf=R['cfg']
    rows=[('初始物源体积',f'{conf.initial_volume_m3:,.0f}','m³'),
          ('释放区长度',f'{conf.source_length_m:.0f}','m'),
          ('释放区宽度',f'{conf.source_width_m:.0f}','m'),
          ('初期摩擦系数 μ₀ ',f'{conf.mu:.4f}','—'),
          ('后期摩擦系数 μ₁',f'{conf.mu_late:.4f}','—'),
          ('速度阻力参数 ξ',f'{conf.xi_mps2:.0f}','m/s²'),
          ('侵蚀系数 k',f'{conf.erosion_k:.6f}','—'),
          ('网格分辨率',f'{conf.grid_m:.0f}','m'),
          ('模拟时长',f'{conf.duration_s:.0f}','s')]
    st.dataframe(pd.DataFrame(rows,columns=['参数','数值','单位']),hide_index=True,width='stretch')
    st.download_button('下载当前参数 JSON',json.dumps(conf.__dict__,ensure_ascii=False,indent=2).encode('utf-8'),
                       file_name='模型参数.json',mime='application/json')
    if st.button('保存本次计算成果',type='secondary'):
        if R.get('precomputed'):
            st.success('预计算成果已位于项目 results 文件夹。')
        else:
            folder=RESULTS/'最新计算'
            with st.spinner('生成栅格、图表与三维网页'):
                save_all(T,R,folder,html=True)
            st.success(f'已保存至 {folder}')
    st.caption('文件输出包括时间序列、三维动画、流深与速度 GeoTIFF、侵蚀和沉积栅格。')
```